

**WARSZAWSKA
WYŻSZA SZKOŁA INFORMATYKI**

PRACA MAGISTERSKA

Konrad Strzechowski

Numer albumu: 10822

**Projekt i implementacja inteligentnego systemu infolinii
medycznej z wykorzystaniem Amazon Connect Customer
i wybranych usług chmurowych AWS**

Promotor:

Dr inż. Waldemar Łabuda

Praca spełnia wymagania stawiane pracom dyplomowym na studiach drugiego stopnia.

WARSZAWA 2026

Spis treści

1 WSTĘP I ROZPOZNANIE DZIEDZINY PROBLEMU

Współczesny system ochrony zdrowia stoi w obliczu wyzwań związanych z zarządzaniem ruchem pacjentów oraz optymalizacją procesów administracyjnych. Wzrost zapotrzebowania na usługi medyczne przy jednoczesnym ograniczeniu kadrowym placówek zdrowotnych sprawiają, że tradycyjne metody obsługi pacjenta stają się niewystarczające. Jednym z najbardziej obciążonych elementów w obszarze komunikacji pozostają infolinie, które stanowią podstawowy kanał kontaktu pacjentów z przychodniami. W rezultacie często występują długie czasy oczekiwania na połączenie z konsultantem oraz wysoki odsetek nieodebranych lub porzuconych połączeń. To bezpośrednio przekłada się na duże opóźnienia w świadczeniu usług dla pacjentów, a w konsekwencji na spadek ich satysfakcji i rosnące niezadowolenie.

Komunikacja telefoniczna w ochronie zdrowia jest szczególnie wymagająca, często ma charakter wrażliwy i nieprzewidywalny. Pacjenci opisują swoje dolegliwości językiem potocznym, często używając nieprecyzyjnych i wieloznacznych sformułowań, co utrudnia ich szybkie i poprawne sklasyfikowanie. Taka specyfika kontaktu znacząco utrudnia wprowadzenie automatyzacji obsługi pacjentów, zwłaszcza w przypadku rozwiązań opartych na sztywnych, predefiniowanych schematach działania. Jednocześnie wiele połączeń dotyczy prostych, powtarzalnych czynności, takich jak umawianie wizyt, odwoływanie konsultacji czy udzielanie podstawowych informacji o funkcjonowaniu placówki. Powoduje to dodatkowe obciążenie personelu, który przez większość czasu zmuszony jest do obsługi dużej liczby rutynowych zgłoszeń, ograniczając ich dostępność dla bardziej złożonych przypadków.

Rozwiązania bazujące na automatyzacji i sztucznej inteligencji są powszechnie wykorzystywane w systemach infolinii w sektorach takich jak bankowość, handel czy telekomunikacja. Mimo to ich zastosowanie w obszarze ochrony zdrowia pozostaje nadal bardzo ograniczone. Wynika to zarówno ze specyfiki komunikacji z pacjentem, jak i z wysokich standardów, którym musi sprostać infrastruktura medyczna. Rozwiązania stosowane w ochronie zdrowia wymagają bowiem najwyższego poziomu niezawodności, całodobowej dostępności oraz rygorystycznego przestrzegania zasad bezpieczeństwa danych i prywatności (np. RODO/GDPR, HIPAA). Doświadczenia innych branż pokazują jednak, że odpowiednio zaprojektowana automatyzacja może realnie poprawić jakość i dostępność usług. W tym kontekście coraz silniej zauważalna staje się potrzeba usprawnienia obsługi pacjentów i odciążenia pracowników.

1.1 Automatyczne systemy infolinii

Zautomatyzowane systemy obsługi klientów odgrywają wielką rolę w modernizacji procesów komunikacyjnych w wielu różnych branżach. Również w placówkach medycznych widoczne jest stosowanie rozwiązań wykorzystujących klasyczne systemy IVR (ang. Interactive Voice Response), oparte na hierarchicznych drzewach menu sterowanych prostymi komendami głosowymi lub tonowymi. Niestety, choć umożliwiają one podstawową automatyzację, coraz częściej okazują się ~~one~~ niewystarczające do efektywnej obsługi złożonych i wielowątkowych zapytań pacjentów. Tradycyjne algorytmy komunikacyjne, oparte na z góry określonych regułach, nie są w stanie prawidłowo zinterpretować niejasno sformułowanych dolegliwości pacjenta ani na ich podstawie poprawnie zidentyfikować zgłaszanego problemu.

W ostatnich latach obserwuje się rozwój bardziej zaawansowanych rozwiązań, określanych jako inteligentne infolinie. Wykorzystują one technologie analizy języka naturalnego oraz rozpoznawania mowy, dzięki czemu potrafią zidentyfikować intencję wypowiedzi, a następnie podjąć odpowiednie działanie, co pozwala na prowadzenie dialogu w sposób zbliżony do rozmowy z konsultantem. Nowa technologia może stanowić odpowiedź na ograniczenia tradycyjnych systemów i rosnące potrzeby placówek.

Warto podkreślić, że inteligentne infolinie nie zastępują personelu medycznego, lecz stanowią narzędzie wspierające jego pracę. Przejmując powtarzalne zadania administracyjne, umożliwiają pracownikom skupienie się na bardziej złożonych przypadkach wymagających indywidualnego podejścia. W efekcie poprawia się zarówno efektywność organizacyjna, jak i jakość obsługi pacjentów, a to czyni automatyzację jednym z kluczowych kierunków rozwoju.

1.2 Cel i zakres pracy

Niniejsza praca magisterska podejmuje tematykę zaprojektowania, implementacji i oceny efektywności inteligentnego systemu infolinii medycznej, uwzględniając jednocześnie ograniczenia oraz zagrożenia związane z jego wdrożeniem, których świadomość jest niezbędna na etapie projektowania. W kontekście zdiagnozowanych problemów charakterystycznych dla medycznych systemów CCaaS (ang. Contact Center as a Service) szczególny nacisk położony będzie na analizę skuteczności algorytmów NLU (ang. Natural Language Understanding) oraz ocenę wydajności architektury opartej na usługach Serverless AWS, stanowiących kluczowe

elementy proponowanego rozwiązania. W ramach tak określonego planu wyróżniono następujące cele szczegółowe:

- analiza i porównanie platform CCaaS dostępnych na rynku,
- opracowanie koncepcji architektonicznej systemu,
- implementacja systemu w postaci PoC (ang. proof of concept),
- przeprowadzenie testów funkcjonalnych oraz ocena efektywności zbudowanego rozwiązania
- porównanie wydajności rozwiązania opartego na NLU z tradycyjnym systemem IVR.

Pierwszym krokiem będzie analiza i porównanie platform CCaaS dostępnych na rynku ze szczególnym uwzględnieniem Amazon Connect Customer oraz Twilio Flex. Zestawienie obejmie ich możliwości, ograniczenia, koszty oraz łatwość wdrożenia w środowisku medycznym. Analiza ta stanowić będzie merytoryczne uzasadnienie wyboru ekosystemu AWS, jako technologicznego fundamentu proponowanego systemu.

Następnie opracowana zostanie koncepcja architektoniczna systemu, obejmująca diagramy komponentów oraz projekt przepływu zdarzeń między Amazon Connect Customer, AWS Lambda i botem Amazon Lex. Wymagania funkcjonalne zostaną przedstawione za pomocą diagramów przypadków użycia i obejmą takie scenariusze, jak: rejestracja wizyt, uzyskiwanie podstawowych danych o placówce czy informowanie pacjenta o wynikach badań. Zdefiniowane zostaną także wymagania poza funkcjonalne, obejmujące skalowalność, bezpieczeństwo danych oraz dostępność i wydajność systemu.

Kolejnym etapem będzie implementacja kompletnego systemu w postaci PoC, obejmująca budowę i konfigurację wszystkich komponentów, w tym, zarówno bota NLU opartego na Amazon Lex V2, jak również tradycyjnego systemu IVR opartego na sztywno zdefiniowanych krokach działania. Oba podejścia zostaną zintegrowane z przykładowym centralnym systemem zarządzania pacjentami.

W ramach weryfikacji poprawności działania systemu przeprowadzone zostaną testy weryfikujące poprawność działania poszczególnych komponentów oraz ich wzajemną integrację. Pozwoli to na ocenę efektywności zaproponowanego rozwiązania w warunkach zbliżonych do rzeczywistych.

Ostatnim krokiem będzie porównanie nowoczesnego podejścia opartego na NLU z tradycyjnym systemem IVR, uwzględniające kryteria takie, jak: trafność rozpoznawania

intencji, czas obsługi zapytania oraz odsetek poprawnie zakończonych interakcji na bazie rzeczywistych wzorców zapytań pacjentów.

1.3 Sformułowanie hipotezy badawczej

W obliczu rosnących potrzeb w zakresie automatyzacji obsługi pacjentów oraz ograniczeń klasycznych systemów IVR konieczne stało się określenie, czy nowoczesne rozwiązania oparte na analizie języka naturalnego mogą realnie poprawić jakość i efektywność komunikacji w placówkach medycznych. Szczególnie istotne jest zbadanie, czy architektura oparta na usługach chmurowych pozwala na budowę systemu skalowalnego, dostępnego i odpornego na zmienne obciążenia. Na tej podstawie sformułowano następującą hipotezę badawczą:

Wykorzystanie ekosystemu usług Serverless AWS umożliwia budowę skalowalnego i inteligentnego systemu infolinii medycznej, który, w porównaniu do klasycznych rozwiązań IVR, wykazuje wyższą skuteczność w rozumieniu i obsłudze zapytań użytkowników, a także zapewnia lepszą skalowalność, dostępność i efektywność operacyjną.

Weryfikacja hipotezy zostanie przeprowadzona poprzez analizę wyników testów funkcjonalnych, porównanie jakości rozpoznawania intencji oraz ocenę wydajności obu podejść w oparciu o rzeczywiste scenariusze komunikacji pacjentów.

2 ANALIZA DZIEDZINY PROBLEMU

Rozdział ten poświęcony jest analizie dziedziny problemu obejmującej kluczowe pojęcia związane z inteligentnymi systemami obsługi zgłoszeń, specyfikę funkcjonowania call centers oraz wyzwania związane z automatyzacją komunikacji w ochronie zdrowia. W dalszej części omówione zostaną technologie chmurowe wykorzystywane przy budowie nowoczesnych infolinii, ze szczególnym uwzględnieniem usług AWS oraz porównaniem platform Amazon Connect Customer i Twilio Flex, co pozwoli określić ich wady i zalety w kontekście projektowanego rozwiązania.

2.1 Definicja kluczowych pojęć i technologii

Celem tego podrozdziału jest przedstawienie kluczowych pojęć, terminów i technologii wykorzystywanych w pracy. Stanowią one będą podstawę do dalszej analizy, implementacji oraz przeprowadzanych badań. Omówione zostaną koncepcje związane z systemami CCaaS, usługami chmurowymi i szeroko pojętą sztuczną inteligencją.

2.1.1 Usługi chmurowe

Usługi chmurowe stanowią fundament współczesnych aplikacji i systemów IT. Umożliwiają wysoką dostępność, elastyczne skalowanie, a dodatkowo pozwalają na minimalizację kosztów infrastruktury i zużycia. W kontekście projektowanego systemu główne znaczenie ma architektura serverless, FaaS (ang. Functions as a Service) oraz IaC (ang. Infrastructure as Code).

Architektura serverless zakłada, że dostawca chmury w pełni odpowiada za zarządzanie infrastrukturą, klient skupia się jedynie na logice uruchamianej aplikacji. FaaS (ang. Functions as a Service) jest kluczowym modelem architektury serverless [1]. Polega na dzieleniu systemu na niewielkie komponenty (krótkie funkcje) uruchamiane automatycznie w reakcji na określone zdarzenia. Takie podejście eliminuje konieczność bezpośredniego utrzymywania serwerów, zapewnia automatyczne skalowanie, upraszcza proces wdrażania oraz pozwala rozliczać się jedynie za faktyczne wykorzystanie zasobów. Warto zaznaczyć, że określenie serverless nie

oznacza całkowitego braku serwerów, lecz brak konieczności ich zarządzania przez użytkownika.

Podejście IaC (ang. Infrastructure as Code) zakłada definiowanie infrastruktury za pomocą kodu, zamiast ręcznej konfiguracji w interfejsie. W projektowanym systemie IaC realizowane jest za pomocą AWS Cloud Development Kit (CDK), co umożliwia tworzenie zasobów AWS w popularnych językach programowania, takich jak TypeScript czy Python [2]. CDK tworzy serwisy w CloudFormation i automatycznie wdraża zdefiniowane zasoby. Zapewnia powtarzalność, kontrolę wersji oraz spójność środowisk, eliminując możliwe błędy wynikające z manualnego wdrażania.

Amazon Connect Customer to chmurowa platforma typu CCaaS, która umożliwia budowę systemów infolinii i Contact Centers. W projektowanym rozwiązaniu stanowi główny punkt kontaktu pacjenta z systemem. Obejmuje obsługę połączeń głosowych, umożliwia tworzenie tradycyjnych przepływów IVR (ang. Contact Flows), integrację z Amazon Lex. Co więcej Amazon Connect zarządza historią połączeń, nagraniami, logami systemowymi, a także pozwala na generowanie raportów bez konieczności stosowania dodatkowych narzędzi [3].

AWS Lambda jest praktyczną realizacją modelu FaaS i jednym z kluczowych komponentów projektowanej infolinii. Funkcje Lambda pozwalają na wykonywanie logiki biznesowej, integrację z innymi komponentami, zarówno innymi wewnętrznymi usługami AWS, jak i zewnętrznymi systemami. Mogą być uruchamiane bezpośrednio z flow Amazon Connect czy Amazon Lex, co umożliwia natychmiastowe reagowanie na działania pacjenta w trakcie rozmowy, bez konieczności utrzymywania aplikacji pozostającej w ciągłej gotowości.

Amazon Lex umożliwia budowę botów opartych na NLU. Odpowiada za, między innymi, analizę wypowiedzi, rozpoznawanie intencji pacjenta, generowanie odpowiedzi, zbieranie danych czy przekazywanie ich do centralnego systemu za pomocą funkcji AWS Lambda.

Amazon DynamoDB to nierelacyjna baza danych NoSQL w pełni zarządzana przez AWS. Umożliwia przechowywanie ogromnych ilości danych, zapewniając przy tym bardzo

niskie opóźnienia odczytu i zapisu oraz efektywne skalowanie dostosowane do aktualnych potrzeb.

Amazon Simple Storage Service (S3) to usługa do przechowywania obiektów o bardzo wysokiej trwałości i dostępności. Może służyć do przechowywania plików, raportów, nagrań rozmów, a także archiwizacji logów. S3 współpracuje z wieloma serwisami AWS bez dodatkowej, skomplikowanej konfiguracji, w tym także z Amazon Connect Customer.

Amazon API Gateway umożliwia tworzenie, publikowanie i zarządzanie interfejsami API wykorzystywanymi przez aplikacje webowe, kontenerowe oraz rozwiązania serverless. W projektowanym systemie będzie pełnił rolę warstwy symulującej centralny system zarządzania danymi medycznymi, co pozwoli odwzorować sposób, w jaki rzeczywiste systemy ochrony zdrowia udostępniają dane poprzez bezpieczne i standaryzowane API.

2.1.2 Sztuczna inteligencja

Sztuczna inteligencja odgrywa krytyczną rolę w nowoczesnych systemach informatycznych, znajdując zastosowanie niemal w każdej dziedzinie. Umożliwia zarówno rozpoznawanie oraz interpretowanie mowy użytkownika, jak i przetwarzanie danych, generowanie odpowiedzi czy automatyczne pozyskiwanie odpowiednich danych. Kluczowym elementem w rozważanej infrastrukturze jest Amazon Lex, który wykorzystując modele uczenia maszynowego rozpoznaje intencje użytkownika, zadaje pytania uzupełniające, weryfikuje dane pacjenta i steruje przebiegiem całej interakcji.

Podstawą działania systemów konwersacyjnych są techniki NLP (ang. Natural Language Processing), obejmujące metody analizy języka naturalnego, takie jak tokenizacja, klasyfikacja tekstu czy analiza składniowa. NLP umożliwia przekształcenie wypowiedzi użytkownika w strukturę, którą system może przetwarzać [4].

NLU (ang. Natural Language Understanding) odpowiada za interpretację znaczenia wypowiedzi. Obejmuje rozpoznawanie intencji użytkownika oraz identyfikację slotów, czyli pojedynczych informacji, takich jak imię, nazwisko czy numer PESEL. Mechanizm ten pozwala systemowi prowadzić dialog w sposób uporządkowany i dostosowany do kontekstu rozmowy [5].

Istotnym elementem systemów głosowych jest również ASR (ang. Automatic Speech Recognition), czyli technologia zamieniająca mowę na tekst, oraz TTS (ang. Text-to-Speech), odpowiedzialna za generowanie mowy na podstawie tekstu. Dzięki nim możliwa jest naturalna komunikacja głosowa z użytkownikiem, bez konieczności korzystania z interfejsów tekstowych.

2.1.3 Systemy CCaaS, IVR i boty konwersacyjne

Systemy obsługi kontaktu z klientem ewoluowały od prostych, statycznych menu głosowych do zaawansowanych platform wykorzystujących sztuczną inteligencję. Współczesne rozwiązania umożliwiają prowadzenie naturalnych dialogów, automatyzację obsługi oraz integrację z systemami zewnętrznymi. Kluczowe pojęcia w tym obszarze to CCaaS, IVR oraz boty konwersacyjne, które stanowią fundament projektowanej infolinii medycznej.

CCaaS (ang. Contact Center as a Service) to model dostarczania systemów call center w formie usługi chmurowej. Umożliwia obsługę połączeń głosowych, czatów i innych kanałów komunikacji bez konieczności utrzymywania własnej infrastruktury. Rozwiązania CCaaS zapewniają skalowalność, wysoką dostępność oraz możliwość integracji z centralnymi systemami, bazami danych czy usługami AI. W kontekście infolinii medycznej CCaaS umożliwia obsługę dużej liczby połączeń oraz automatyzację części procesów [6].

Tradycyjne systemy IVR (ang. Interactive Voice Response) opierały się na statycznych menu, w których użytkownik wybierał opcje za pomocą klawiatury telefonu. Współczesne IVR wykorzystują rozpoznawanie mowy, co pozwala na bardziej naturalną interakcję, jednak nadal opierają się na z góry zdefiniowanych ścieżkach. Ich funkcjonalność jest ograniczona w porównaniu z systemami opartymi na sztucznej inteligencji.

Najbardziej zaawansowaną formą automatyzacji są boty konwersacyjne, które wykorzystują techniki NLP i NLU do prowadzenia dialogu w sposób zbliżony do rozmowy z człowiekiem. Boty te potrafią rozpoznawać intencje użytkownika, zadawać pytania uzupełniające, analizować kontekst oraz podejmować decyzje w oparciu o przekazywane informacje. Szczegółowe porównanie obu podejść obsługi klienta przedstawia **Tabela 1.1**.

Tabela 1.1: Analiza porównawcza klasycznych systemów IVR i botów konwersacyjnych [7].

Cecha	IVR	Bot konwersacyjny
Sposób interakcji	Statyczne menu, wybór cyfr.	Język naturalny, swobodna rozmowa.
Elastyczność dialogu	Niska, z góry ustalone ścieżki.	Wysoka, dynamiczne dopasowanie do kontekstu.
Wielojęzyczność	Wymaga osobnych drzew menu na każdy język.	Bot Locale per język; współdzielona logika biznesowa.
Zrozumienie wypowiedzi	Tylko proste komendy.	NLP + NLU, rozpoznawanie intencji i slotów.
Obsługa błędów	Ograniczona („wybierz ponownie”).	Inteligentne dopytywanie i korekta.
Wrażenia użytkownika	Mechaniczne, nienaturalne.	Naturalne, zbliżone do rozmowy z człowiekiem.
Zastosowanie	Proste procesy.	Złożone procesy wymagające dialogu.
Integracja z danymi	Minimalna.	Pełna możliwość pobierania i walidacji danych.

Omówione zagadnienia stanowią podstawę do dalszych rozważań dotyczących zastosowań rozwiązań CCaaS w systemach ochrony zdrowia.

2.2 Charakterystyka CCaaS w systemach ochrony zdrowia.

Systemy CCaaS (ang. Contact Center as a Service) stanowią rozwinięcie tradycyjnych rozwiązań call center, przenosząc infrastrukturę komunikacyjną do chmury obliczeniowej. W przeciwieństwie do systemów lokalnych (ang. on-premise), model CCaaS eliminuje konieczność utrzymywania własnego sprzętu i oprogramowania, oferując skalowalne środowisko dostępne w formie subskrypcji. Placówki medyczne mogą dzięki temu elastycznie dostosowywać przepustowość systemu do bieżącego obciążenia, bez konieczności inwestowania w dodatkową infrastrukturę.

W ochronie zdrowia systemy CCaaS muszą jednak spełniać wymagania wykraczające poza typowe zastosowania komercyjne. Komunikacja z pacjentami dotyczy często kwestii wrażliwych, co wymaga wysokiej precyzji działania oraz odpowiedniego dostosowania algorytmów przetwarzania języka naturalnego. Placówki medyczne funkcjonują w ramach rygorystycznych regulacji dotyczących danych osobowych i medycznych. Na poziomie krajowym istotne znaczenie ma Ustawa z dnia 28 kwietnia 2011 r. o systemie informacji w ochronie zdrowia [8], która określa zasady gromadzenia, przetwarzania i udostępniania danych medycznych. Dodatkowo prawa unijne wyznacza RODO/GDPR [9], uzupełnione od października 2024 roku przez dyrektywę NIS2 [10], obejmującą rygorystyczne wymagania w zakresie cyberbezpieczeństwa, zarządzanie ryzykiem, zgłaszanie incydentów i zapewnienie ciągłości działania. W przypadku rozwiązań adresowanych do rynku amerykańskiego zastosowanie ma ustawa HIPAA [11]. Konsekwencją tak złożonego środowiska prawnego jest konieczność stosowania mechanizmów zapewniających poufność i integralność danych oraz spełnienia wymogów dotyczących ich przetwarzania i przechowywania.

Wdrożenie CCaaS w środowisku medycznym wiąże się również z wyzwaniami technicznymi. Jednym z nich jest integracja z istniejącymi systemami informatycznymi placówki, takimi jak HIS (ang. Hospital Information System) czy EMR (ang. Electronic Medical Records). Systemy te często oparte są na starszych technologiach i nie udostępniają nowoczesnych interfejsów API, co utrudnia wymianę danych [12]. Konieczne jest zatem stworzenie warstwy integracyjnej umożliwiającej spójne przekazywanie informacji między systemem infolinii a bazą danych pacjentów, kalendarzem wizyt czy modułami obsługi wyników badań.

Kolejnym kluczowym aspektem jest wymóg wysokiej dostępności i niezawodności. W ochronie zdrowia nawet krótkotrwała niedostępność systemu komunikacyjnego może mieć bezpośrednie konsekwencje dla bezpieczeństwa pacjentów, zwłaszcza w przypadku zgłoszeń pilnych. Dlatego rozwiązania stosowane w tym sektorze muszą zapewniać gwarantowany poziom dostępności (SLA) na poziomie co najmniej 99,9%, co odpowiada maksymalnie kilku godzinom przestoju rocznie.

Wdrożenie CCaaS wiąże się również z wyzwaniami organizacyjnymi. Personel medyczny i administracyjny, przyzwyczajony do tradycyjnych metod obsługi telefonicznej, może początkowo wykazywać opór wobec nowych rozwiązań, zwłaszcza tych opartych na automatyzacji. Również pacjenci mogą odczuwać niepewność w kontakcie z systemem automatycznym. Skuteczne wdrożenie wymaga więc odpowiedniego zarządzania zmianą, szkoleń oraz jasnej komunikacji z użytkownikami końcowymi.

2.3 Analiza porównawcza platform CCaaS: Amazon Connect Customer vs Twilio Flex

Wybór platformy CCaaS determinuje architekturę całego systemu infolinii. Decyzja ta wpływa na dostępne mechanizmy integracji z zewnętrznymi systemami medycznymi, możliwości wbudowanej sztucznej inteligencji, podejście do bezpieczeństwa danych oraz model rozliczeniowy. Niniejszy podrozdział zawiera szczegółową analizę porównawczą dwóch czołowych platform dostępnych na rynku w 2026 roku: Amazon Connect Customer oraz Twilio Flex.

2.3.1 Amazon Connect Customer

Amazon Connect Customer to w pełni zarządzana platforma Contact Center opracowana przez AWS i wprowadzona na rynek w 2017 roku. Jest ściśle zintegrowana z ekosystemem chmurowym Amazonu, co umożliwia bezpośrednie wykorzystanie takich usług jak AWS Lambda, Amazon Lex, Amazon S3 czy Amazon CloudWatch bez konieczności dodatkowej konfiguracji.

Connect udostępnia narzędzia do tworzenia tradycyjnych przepływów rozmów IVR w graficznym edytorze, dodatkowo rozszerzając je o zaawansowaną logikę biznesową poprzez wywoływanie funkcji AWS Lambda oraz integrację z botami Amazon Lex. Oprócz natywnej obsługi połączeń głosowych i transkrypcji, Connect zapewnia także automatyczne przetwarzanie danych i generowanie raportów. Dzięki temu platforma wspiera jednocześnie pacjentów korzystających z infolinii, agentów obsługujących połączenia, jak i kadrę zarządzającą, której dostarcza narzędzi do monitorowania i raportowania całego systemu Contact Center.

Platforma opiera się na modelu event-driven, w którym każde działanie systemu bazuje na odpowiednim zdarzeniu. Pozwala to na pełne wykorzystanie modelu kosztów AWS PAYG (ang. pay-as-you-go), w którym wszystkie opłaty wynikają wyłącznie z realnego zużycia zasobów, a nie utrzymywania nieaktywnej infrastruktury systemu. W momencie opracowywania niniejszej pracy koszt połączeń w Amazon Connect wynosi \$0,038 za każdą minutę połączenia, powiększony o dodatkową opłatę zależną od regionu oraz metody komunikacji (dla połączeń przychodzących w Polsce: \$0.005) [13], przy czym nie są naliczane żadne opłaty licencyjne za korzystanie z systemu przez agentów. W kwestii wydatków należy jednak pamiętać o równie istotnych opłatach za korzystanie z pozostałych usług AWS. Każde wykorzystanie AWS Lambda, przechowanie danych w Amazon DynamoDB czy obsługa bota

Amazon Lex generuje koszty, z którymi trzeba się liczyć. Oznacza to, że wykonywanie skomplikowanej logiki biznesowej lub gromadzenie dużej ilości danych dla każdego pacjenta czy połączenia może prowadzić do znacznego wzrostu wydatków przy skalowaniu rozwiązania.

Amazon Connect Customer spełnia standardy regulacyjne istotne dla sektora medycznego. Usługa zapewnia zgodność z HIPAA, a AWS pozwala na zawarcie umowy BAA (ang. Business Associate Agreement). Dodatkowo infrastruktura AWS posiada certyfikacje takie jak PCI DSS, SOC 1/2/3 oraz ISO 27001 [14], potwierdzające stosowanie zaawansowanych mechanizmów ochrony.

Dla wdrożeń realizowanych na potrzeby polskiego sektora ochrony zdrowia podstawowe znaczenie ma możliwość korzystania z regionu EU (Frankfurt) oraz konfiguracji, która gwarantuje, że przetwarzane dane nie opuszczą obszaru EOG (Europejski Obszar Gospodarczy). Takie podejście ułatwia realizację obowiązków wynikających z RODO, zwłaszcza w zakresie ograniczenia transferu danych poza Unię Europejską oraz zapewnienia właściwego poziomu zabezpieczeń

2.3.2 Twilio Flex

Twilio Flex wprowadzony w 2018 roku, reprezentuje odmienną filozofię architektury CCaaS. Działa jako warstwa zarządzania nad infrastrukturą telekomunikacyjną Twilio (Super Network), a więc nie narzuca wybranego środowiska chmurowego, co pozwala klientom na elastyczne dopasowanie kompozycji technologicznej rozwiązania.

Centralnym elementem Twilio Flex jest TaskRouter, czyli silnik trasowania zadań bazujący na regułach przydziału pracy [15]. Umożliwia zarówno obsługę połączeń, ale też wiadomości SMS, a nawet WhatsApp, e-mail oraz czatu. Interfejs agenta jest aplikacją React.js, w pełni konfigurowalną i możliwą do dostosowania do własnych potrzeb za pomocą dedykowanych pluginów. W efekcie integracja interfejsu z dziedzinowymi systemami medycznymi może przebiegać sprawniej. Poza tym ograniczeniem Twilio Flex jest brak natywnego silnika NLU. Dostępne są integracje platformy z zewnętrznymi dostawcami AI, jednak wiąże się to z wprowadzeniem dodatkowych zależności i ryzykiem opóźnień w komunikacji.

Model cenowy Twilio Flex również różni się od modelu AWS. Oparty jest na aktywnych użytkownikach, z opłatą \$1 za godzinę aktywnej pracy agenta lub stałym abonamentem \$150 miesięcznie za agenta [16]. W środowiskach o dużej rotacji (np. kampanie

sezonowe) model godzinowy może być korzystny, jednak w sektorze ochrony zdrowia wysoko dostępna i niezawodna ciągłość obsługi są wymagane przez cały rok. Jak już wspomniano, Twilio nie dostarcza własnego środowiska chmurowego, dlatego pojawiają się dodatkowe koszty wynikające z konieczności utrzymania infrastruktury u wybranego dostawcy lub on-prem, a także wydatki związane z integracją z zewnętrznym dostawcą NLU.

2.3.3 Porównanie i uzasadnienie wyboru platformy

Tabela 1.2 zawiera szczegółowe porównanie kryteriów obu systemów, istotnych zarówno dla typowych wdrożeń, jak i z perspektywy zastosowania w środowisku medycznym. Analiza ta stanowi kluczową podstawę do wyboru bazowej platformy dla niniejszego projektu.

Tabela 1.2: Analiza porównawcza platform Amazon Connect Customer i Twilio Flex

[źródło: opracowanie własne].

Kryterium	Amazon Connect Customer	Twilio Flex
Ekosystem integracji	Natywny dostęp do S3, Lambda, DynamoDB, CloudWatch, SNS, SES, itp.	Bogata oferta marketplace; integracje przez REST API i WebHooks.
Skalowalność	Automatyczna skalowalność AWS; brak limitów pojemności w warstwie IVR.	Skalowanie przez elastyczność API; wymaga konfiguracji Flex Insights.
Integracja AI/NLU	Natywna: Amazon Lex (wbudowany bot NLU/ASR), Comprehend, Polly.	Zewnętrzna: IBM Watson, Dialogflow, dedykowane API; brak natywnego NLU.
Bezpieczeństwo / Zgodność z regulacjami	HIPAA, PCI DSS, SOC 1/2/3, ISO 27001, FedRAMP.	HIPAA, PCI DSS, SOC 2; mniejszy zakres certyfikatów rządowych.
Model cenowy	Pay-as-you-go: ~\$0,038/min połączenia przychodzącego; opłaty za usługi AWS.	opłata \$150/mies za aktywnego agenta; opłaty za utrzymanie infrastruktury i zewnętrzne silniki NLU.

Kryterium	Amazon Connect Customer	Twilio Flex
Wsparcie języka polskiego	Amazon Polly + Lex obsługuje j. polski (TTS/ASR).	Brak natywnego NLU; obsługa wyłącznie poprzez integrację z zewnętrznym dostawcą NLU.
Czas wdrożenia PoC	Kilka godzin; konfiguracja przez interfejs GUI i CloudFormation.	Kilka dni; wymagana konfiguracja kodu TaskRouter, Flex Plugins.
Wsparcie telefonii	Numery w 100+ krajach.	Numery w 100+ krajach.
Personalizacja UI agenta	Amazon Connect Workspace (ograniczona personalizacja).	Flex UI (React), pełna personalizacja komponentów.
Monitoring i SLA	CloudWatch, Contact Lens (analiza konwersacji AI).	Flex Insights, Segment; bardziej otwarte API analityczne.
Poziom zależności od dostawcy	Wysoki, bardzo silna zależność od ekosystemu AWS.	Niższy, agnostyczne API, dowolność infrastruktury i silników AI.

Na podstawie powyższej analizy porównawczej wybrano Amazon Connect Customer jako platformę bazową dla niniejszej pracy. Kluczowym czynnikiem, który zaważył na tej decyzji, jest natywna integracja z całym ekosystemem AWS. Całkowicie eliminuje to potrzebę konfigurowania dodatkowych połączeń API, umożliwiając budowę kompletnego rozwiązania w obrębie jednego dostawcy. Dzięki silnemu powiązaniu z serwisami takimi jak Lambda, DynamoDB czy S3, możliwe jest łatwe tworzenie zarówno prototypu PoC, jak i pełnego systemu produkcyjnego.

Kolejnym istotnym elementem jest natywna integracja z Amazon Lex. Rozwiązanie to znacząco upraszcza cały proces korzystania z botów konwersacyjnych i redukuje opóźnienia, w rozwiązaniu w którym każda sekunda ciszy może stanowić o złym doświadczeniu klienta.

Dodatkowo automatyczna skalowalność infrastruktury AWS pozwala obsługiwać zarówno niewielkie obciążenia testowe, jak i duży ruch produkcyjny bez konieczności ręcznej konfiguracji zasobów. W połączeniu z modelem pay-as-you-go stanowi szczególnie korzystne

rozwiązanie w kontekście akademickiego PoC, umożliwiając kontrolę kosztów przy minimalnym budżecie.

Zgodność z wymaganiami HIPPA oraz możliwość BAA sprawiają, że Amazon Connect oferuje pełną ścieżkę zgodności regulacyjnej dla danych medycznych w ramach jednego dostawcy, gwarantując bezpieczeństwo, integralność i zgodność przetwarzania danych z obowiązującymi standardami i przepisami.

2.4 Analiza techniczna usług AWS

W tym podrozdziale przeprowadzona została szczegółowa analiza najważniejszych komponentów AWS wykorzystanych przy budowie projektowanego systemu. Omówione zostały aspekty techniczne wybranych usług, ich przeznaczenie, metoda zastosowania i parametry konfiguracyjne. Analiza obejmuje również aspekty biznesowy, w szczególności wpływ na wymagania poza funkcjonalne, takie jak koszty, bezpieczeństwo czy skalowalność. Wszystkie przedstawione w pracy koszty i modele rozliczeniowe odnoszą się do regionu eu-central-1 (Frankfurt), który stanowi punkt odniesienia dla projektowanej architektury.

2.4.1 Amazon Connect Customer

Amazon Connect Customer jest kluczową usługą AWS umożliwiającą budowanie skalowalnego, wysoko dostępnego i niezawodnego systemu CCaaS. Instancja Amazon Connect stanowi pełen pakiet konfiguracyjny, obejmujący wszystkie elementy niezbędne do obsługi klientów. Choć domyślnie jedno konto AWS pozwala na posiadanie dwóch instancji Connect, w praktyce jedna w zupełności wystarczy do stworzenia pełnego systemu produkcyjnego. Dodatkowe instancje wykorzystywane są najczęściej, jako środowiska deweloperskie lub testowe, co ułatwia bezpieczne wdrażanie zmian.

Instancja Amazon Connect pełni rolę centralnego punktu systemu Contact Center. Umożliwia tworzenie i zarządzanie przepływami rozmów, konfigurację kolejek, definiowanie uprawnień użytkowników, odbieranie numerów telefonicznych, generowanie raportów oraz monitorowanie historii połączeń, ich szczegółów i nagrań. System pozwala na precyzyjną granulację dostępu, dzięki czemu agenci, nadzorcy i administratorzy mogą otrzymać dokładnie takie uprawnienia, jakich potrzebują.

Contact Flows Editor to graficzny edytor przepływów IVR służący do obsługi połączeń. Jest kluczowym narzędziem wykorzystywanym do projektowania i tworzenia szczegółowej

logiki rozmowy [17]. Zdefiniowane w nim zostają kolejne kroki wykonywane podczas połączenia zależne od zdarzeń i danych wejściowych. Głównymi blokami są między innymi:

- Play prompt – odtworzenie komunikatu dla klienta,
- Store customer input – pobranie danych od klienta,
- AWS Lambda Function – wywołanie funkcji Lambda,
- Set working queue – ustawienie kolejki, używanej podczas transferu,
- Set whisper flow – konfiguracja komunikatu dla klienta/agenta przed połączeniem,
- Transfer to queue – przekierowanie do kolejki agentów,
- Transfer to phone number – przekierowanie na numer zewnętrzny,
- Transfer to flow – przejście do innego przepływu,
- Disconnect – rozłączenie połączenia.

Kluczowym parametrem wydajnościowym jest liczba jednoczesnych aktywnych połączeń. Domyślny limit wynosi 10 współbieżnych połączeń na instancję, jednak jest to jedynie tzw. soft limit. Można go zwiększyć za zgłoszenia do AWS Support. W przypadku wdrożeń dla dużych systemów, takich jak rozpatrywane Contact Center w sektorze ochrony zdrowia, limity rzędu 1000-2000 aktywnych połączeń są standardem. Warto podkreślić, że limity są rozdzielone na różne typy interakcji z różnymi granicami (połączenia przychodzące, czat, email, zadania). Dodatkowo istnieje limit liczby numerów telefonicznych przypisanych do instancji, który domyślnie wynosi 5, lecz również może zostać podniesiony.

Warto również zwrócić uwagę na limity związane z usługą Contact Lens, która odpowiada za analizę rozmów. Domyślnie system pozwala na przetwarzanie 300 równoległych połączeń w czasie rzeczywistym (ang. real-time analytics), oraz 200 w przypadku analizy po ukończeniu rozmowy (ang. post-call analytics). Podsumowanie limitów platformy Amazon Connect Customer opisuje **Tabela 1.3**.

Tabela 1.3: Zestawienie najważniejszych ograniczeń platformy Amazon Connect [18].

Parametr	Domyślny limit	Poziom limitu	Możliwość zwiększenia
Liczba instancji Connect	2	Konto	Tak

Parametr	Domyślny limit	Poziom limitu	Możliwość zwiększenia
Współbieżne połączenia przychodzące	10	Instancja	Tak
Współbieżne sesje czatu	500	Instancja	Tak
Współbieżne sesje email	1000	Instancja	Tak
Liczba numerów telefonicznych	5	Konto	Tak
Liczba użytkowników	500	Instancja	Tak
Liczba AWS Lambda	50	Instancja	Tak
Liczba botów Lex	70	Instancja	Nie
Liczba botów Lex V2	100	Instancja	Tak
Liczba Data Tables	100	Instancja	Tak
Klucze główne Data Tables	5	Data Table	Nie
Liczba atrybutów data table	100	Data Table	Nie
Maksymalna liczba Contact Flows	1000	Instancja	Tak
Contact Lens – analiza real-time	300	Konto	Tak
Contact Lens – analiza post-call	200	Konto	Tak

Mając podsumowane najważniejsze limity głównego komponentu platformy Amazon Connect Customer, można skupić się na kolejnych elementach składających się na projektowany system CCaaS.

2.4.2 Amazon Lex V2 i Amazon Polly

Amazon Lex V2 oraz Amazon Polly stanowią kluczowe komponenty AI wykorzystane przy budowie niniejszego systemu Contact Center. Amazon Lex V2 wprowadzony został w 2021 roku, jako następca Lex V1. Pełni funkcję silnika NLU/ASR, umożliwiając transkrypcję mowy, rozpoznawanie intencji użytkownika, ekstrakcję danych oraz zarządzanie dialogiem. Wersja V2 wprowadziła istotne architektoniczne zmiany w podejściu do budowania botów dodając ulepszone wersjonowanie i precyzyjniejsze modele językowe.

Amazon Polly uzupełnia Lex, dostarczając syntezę mowy TTS (ang. Text-To-Speech). Pozwala na generowanie naturalnie brzmiących komunikatów. Dzięki dodatkowej obsłudze SSML możliwe jest precyzyjne sterowanie tonem, dodawanie przerw, a nawet kontrolowanie akcentu, co poprawia jakość interakcji z użytkownikiem.

2.4.3 AWS Lambda

AWS Lambda wprowadzona w 2014 roku była jedną z pierwszych usług serverless na rynku, zmieniając sposób budowania aplikacji event-driven i stając się fundamentem nowoczesnych architektur chmurowych oraz modelu pay-as-you-go. AWS Lambda pełni rolę środowiska wykonawczego FaaS. Odpowiada za realizację logiki biznesowej oraz integrację z pozostałymi serwisami systemu. Dzięki możliwości uruchamiania kodu w popularnych językach, takich jak TypeScript czy Python, Lambda pozwala na implementację praktycznie dowolnych operacji. Funkcje mogą korzystać z AWS SDK (ang. Software Development Kit) w celu komunikacji z innymi usługami AWS lub integrować się z zewnętrznymi systemami, m.in. poprzez zapytania HTTP.

Podczas korzystania z AWS Lambda klient płaci wyłącznie za liczbę wywołań funkcji i jej czas działania, zależny od przydzielonej pamięci. W ramach usługi dostępna jest także dodatkowo płatna opcja Provisioned Concurrency, która gwarantuje natychmiastową gotowość funkcji za stałą opłatę za utrzymywanie zasobu w gotowości. Natomiast do najważniejszych limitów AWS Lambda należą

- Maksymalny czas wykonywania – 15 minut dla standardowych wywołań. W przypadku wywołań z Amazon Connect Contact Flows czas ten jest znacząco ograniczony, aby zapewnić płynny przebieg rozmowy:
 - 8 sekund dla synchronicznych wywołań,
 - 60 sekund dla asynchronicznych wywołań.
- Równoległe wywołania – domyślny soft-limit wynosi 1000 równoległych uruchomień.

- Limit pamięci – od 128 MB do 10 GB na pojedynczą funkcję.

2.4.4 Amazon DynamoDB

Amazon DynamoDB jest w pełni zarządzaną, serverless bazą danych NoSQL oferowaną przez AWS. Została zaprojektowana z myślą o niskich opóźnieniach, wysokiej dostępności i przepustowości. Dzięki wsparciu trybów provisioned oraz on-demand, baza automatycznie skaluje wydajność operacji odczytu RCU (ang. Read Capacity Unit) oraz zapisu WCU (ang. Write Capacity Unit) do aktualnego obciążenia. Pozwala to na optymalizację kosztów, a klienci płacą tylko za liczbę wykonanych zapytań oraz przechowywanie i transfer danych. DynamoDB może służyć do zarówno przechowywania danych o pacjentach, jak i do gromadzenia logów czy utrzymywania danych pomocniczych wykorzystywanych przez funkcje Lambda.

2.4.5 Amazon S3

Amazon S3 (ang. Simple Storage Service) pełni funkcję skalowalnego magazynu obiektów. Początkowo był stosowany głównie do zwykłego przechowywania plików, jednak z czasem stał się fundamentem usług AWS wykorzystywanym do wielu architektur. W kontekście rozpatrywanego systemu Contact Center pozwala na przechowywanie nagrań rozmów, logów, raportów, artefaktów CI/CD i plików konfiguracyjnych.

S3 zapewnia wysoką trwałość danych (11 dziewiątek) oraz różne klasy przechowywania, stosowane dla różnych zastosowań, od danych często używanych po archiwa wymagane przez regulacje prawne. S3 umożliwia także definiowanie polityk retencji i automatyczne przenoszenie starszych obiektów między klasami.

Do istotnych ograniczeń należą maksymalny rozmiar pojedynczego obiektu (5 TB) oraz limit wielkości danych przesyłanych w jednym żądaniu PUT. Warto podkreślić, że S3 nie posiada tradycyjnej struktury folderów. Wszystkie pliki są przechowywane jako obiekty z kluczami, które jedynie symulują strukturę katalogów. Każdy obiekt może być przechowywany w innej klasie storage, nawet jeśli znajduje się w tym samym buckecie. Koszty korzystania z S3 zależą od wybranej klasy storage, ilości przechowywanych danych oraz liczby operacji wykonywanych na obiektach.

2.4.6 Amazon CloudWatch

Amazon CloudWatch to scentralizowana usługa do monitorowania zasobów i aplikacji w środowisku chmurowym AWS. Jej głównym zadaniem jest gromadzenie logów (m.in. z usług Amazon Connect i AWS Lambda), rejestrowanie zdarzeń systemowych oraz tworzenie metryk operacyjnych. Pozwala to na skuteczne debugowanie błędów, przeprowadzenie analizy wydajności czy pomiary opóźnień. Ponadto CloudWatch umożliwia tworzenie alarmów integrujących się z Amazon SNS, pozwalając na wysyłanie powiadomień o zdarzeniach wymagających natychmiastowej interwencji.

2.4.7 AWS IAM

AWS IAM (ang. Identity and Access Management) to kluczowa usługa do zarządzania tożsamością i uprawnieniami w środowisku AWS. Odpowiada za precyzyjną, granularną kontrolę dostępu zarówno dla wszystkich użytkowników chmury (np. developerów), jak i wszystkich komponentów systemu. AWS silnie zaleca stosowanie zasady najmniejszych przywilejów (ang. Principle of Least Privilege), zgodnie z którą każdy użytkownik i komponent powinien posiadać wyłącznie uprawnienia do zasobów, które są mu niezbędne. Oznacza to, że każdy podmiot korzysta ze zdefiniowanej roli IAM, nadającej mu uprawnienia do wybranych działań. Przykładowo, Lambda współpracująca z Amazon DynamoDB powinna mieć dostęp tylko do właściwej tabeli, bez możliwości ingerencji w inne usługi, takie jak Amazon S3. Co ważne, IAM jest usługą całkowicie darmową. Koszty związane są jedynie z wykorzystywanymi zasobami, natomiast samo zarządzanie tożsamością i dostęпами nie generuje opłat.

2.4.8 Podsumowanie usług AWS

Podsumowując, wymienione powyżej usługi pozwalają na budowę bezpiecznego i niezawodnego systemu Contact Center. Wykorzystanie architektury serverless gwarantuje wysoką dostępność i automatyczną skalowalność w czasie rzeczywistym oraz elastyczność kosztową wynikającą z modelu pay-as-you-go. Sprzyja to sprawnej budowie prototypów PoC, jednocześnie umożliwiając płynne przejście do usystematyzowanych wdrożeń produkcyjnych. **Tabela 1.4** zestawia najbardziej istotne funkcje oraz opłaty dla każdej z opisanych usług.

Tabela 1.4: Rola usług AWS w architekturze systemu CCaaS [19].

Usługa AWS	Warstwa architektury	Kluczowa odpowiedzialność	Główne Oplaty
Amazon Connect Customer	Platforma CCaaS	Punkt startowy interakcji z pacjentem, obsługa połączeń, routing, raportowanie;	Pay-as-you-go: ~\$0,038/min połączenia przychodzącego
Amazon Lex	Silnik NLU/ASR	Rozpoznawanie intencji, generowanie odpowiedzi, zarządzanie rozmową	\$0.004 za każde zapytanie głosowe; \$0.0065 za każde 15 sekund strumieniowania
Amazon Polly	Usługa TTS	Generowanie mowy dla odpowiedzi głosowych	\$4/1 mln znaków (Standard), \$16/1mln znaków (Neural)
AWS Lambda	Środowisko wykonawcze (FaaS)	Przetwarzanie danych, logika biznesowa, integracja z systemami wewnętrznymi i zewnętrznymi	\$0,0000166667 / GB-s; \$0,20 / 1 mln żądań
Amazon DynamoDB	Baza danych NoSQL	Przechowywanie danych	Zapis: \$0,7625 / mln WRU; Odczyt: \$0,1525 / mln RRU; Przechowywanie: \$0,306 / GB-mies.
Amazon S3	Magazyn obiektów	Przechowywanie plików, logów, raportów	Przechowywanie: \$0,0245 / GB; Operacje PUT/COPY/POST/LIST: \$0,0054 / 1000 żądań; GET/SELECT: \$0,00043 / 1000 żądań
Amazon CloudWatch	System obserwowalności	Monitorowanie systemu, logi, metryki, alerty	Metryki: \$0,63 / GB-mies.; Logs Insights: \$0,0063 / GB

Usługa AWS	Warstwa architektury	Kluczowa odpowiedzialność	Główne Oplaty
			przeskanowanych danych
AWS IAM	System zarządzania tożsamością i dostępem	Zarządzanie dostępem, definiowanie ról, uprawnień, polityk	Bezpłatna

Zestawienie to stanowi pełną charakterystykę usług wykorzystywanych w architekturze systemu CCaaS i wyznacza fundament do dalszego projektowania logiki połączeń.

2.5 Specyfikacja wymagań i parametry pomiarowe

Projektowanie złożonych systemów informatycznych, takich jak rozpatrywana platforma Contact Center dla sektora ochrony zdrowia, wymaga w pierwszej kolejności szczegółowego zdefiniowania wymagań funkcjonalnych i poza funkcjonalnych. Na podstawie przeprowadzonej analizy opiera się fundament całego modelowania systemu oraz jego późniejsza implementacja. Należy podkreślić, że wprowadzanie zmian w uprzednio zatwierdzonych wymaganiach wiąże się z wykładniczym wzrostem kosztów względem etapu projektu, dlatego tak istotne jest ich dokładne przemyślenie.

Wymagania biznesowe oraz główne funkcje systemu zostaną zaprezentowane za pomocą przypadków użycia, które precyzyjnie opiszą cały przebieg rozmowy z perspektywy użytkownika (pacjenta lub agenta). Następnie określone zostaną mierzalne metryki sukcesu, które stanowić będą podstawę do weryfikacji rozwiązania i budowy w pełni produkcyjnego systemu.

2.5.1 Wymagania funkcjonalne

Poniższa sekcja przedstawia zebrane i pogrupowane tematycznie wymagania funkcjonalne (tabele 2.5-2.8), definiujące najważniejsze działania systemu CCaaS z perspektywy pacjenta, agenta i samego systemu. Obejmują one między innymi udostępnianie podstawowych informacji o placówkach, rejestrację oraz odwoływanie wizyt. W niniejszej pracy wymienione zostały jedynie główne elementy funkcjonalne objęte zakresem

przygotowywanego PoC, stanowiące podstawę do dalszego rozwoju. Wymagania przedstawione są w postaci tabel, w których kolumny opisują kolejno kto i jaką akcję może wykonać w systemie oraz jaki jest zakładany wynik wykonania tej akcji. W celu ustalenia priorytetu każdego z wymagań wykorzystano metodę MoSCoW, w której wyróżniamy 4 następujące kategorie, określające priorytet danego zadania: M – MUST (musi być), S – SHOULD (powinien być), C – COULD (może być), W – WON'T (nie będzie).

Tabela 1.5: Wymagania dotyczące obsługi połączeń przychodzących [źródło: opracowanie własne].

Aktor	Akcja	Wynik
Pacjent	Użycie globalnej komendy (np. transfer, powtórzenie) [M]	System interpretuje komendę niezależnie od aktualnego etapu rozmowy i wykonuje odpowiednią akcję
	Zażądanie kontaktu z agentem [M]	System interpretuje komendę transferu i przekierowuje pacjenta do kolejki
	Poproszenie o powtórzenie informacji [S]	System ponownie odtwarza ostatni komunikat głosowy
	Utworzenie konta w systemie [M]	Pacjent rejestruje się podając wymagane dane, a system tworzy jego konto
	Uwierzytelnienie [M]	Pacjent podaje wymagane dane, a system weryfikuje jego tożsamość
System	Brak zrozumienia intencji pacjenta. [M]	Po trzykrotnej nieudanej próbie interpretacji wypowiedzi pacjenta, system transferuje połączenie do agenta
	Nieobsługiwany błąd systemu [M]	System przekierowuje połączenie do agenta.
	Zakończenie rozmowy [S]	System zapisuje podstawowe informacje o przebiegu rozmowy w ceku analizy

Tabela 1.6: Wymagania dotyczące informacji ogólnych funkcjonowania placówki [źródło: opracowanie własne].

Aktor	Akcja	Wynik
Pacjent	Uzyskanie podstawowych informacji o placówce (adres, godziny pracy) [M]	Odtworzony zostaje komunikat z prawidłową odpowiedzią
	Uzyskanie informacji o dostępnych usługach [S]	Odtworzony zostaje komunikat z listą dostępnych usług
	Uzyskanie informacji o lokalizacji i dojazdach [C]	Odtworzony zostaje komunikat z lokalizacją i dojazdami z najpopularniejszych miejsc

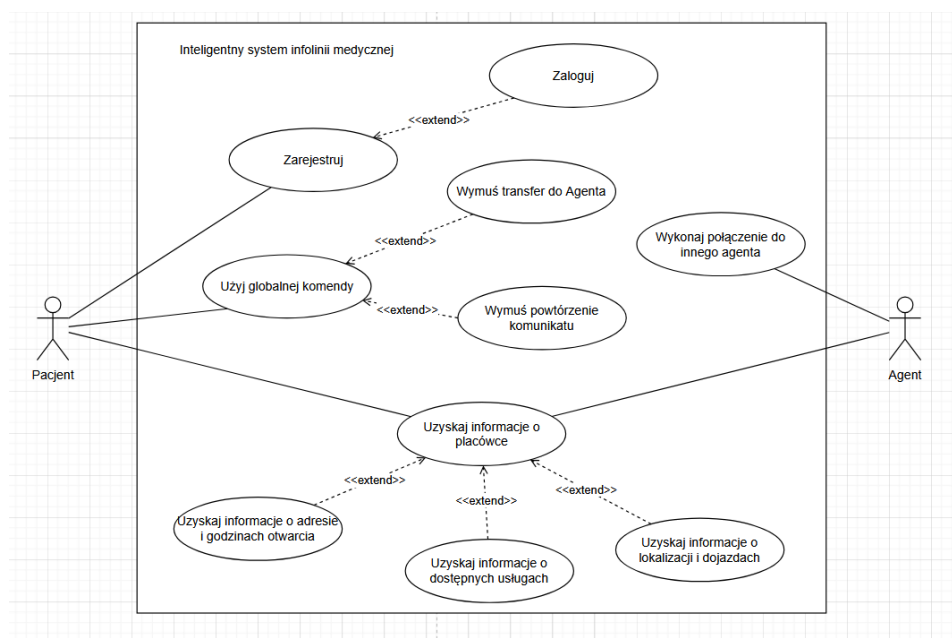
Tabela 1.7: Wymagania dotyczące obsługi pacjenta [źródło: opracowanie własne].

Aktor	Akcja	Wynik
Uwierzytelniony Pacjent	Zarejestrowanie nowej wizyty u lekarza [M]	System sprawdza dostępność terminów, rezerwuje wybrane okno czasowe i potwierdza operację z pacjentem
	Odczytanie listy zaplanowanych wizyt [M]	System pobiera listę zaplanowanych wizyt pacjenta i odtwarza ich szczegóły
	Odwołanie zaplanowanej wizyty [M]	System identyfikuje wizytę, anuluje ją i zwalnia termin dla innych użytkowników
	Przełożenie terminu zarezerwowanej wizyty [S]	System identyfikuje wizytę, rezerwuje nowo wybrane okno czasowe i zwalnia poprzedni termin innym użytkownikom
	Uzyskanie informacji o danych pacjenta [M]	System odtwarza dane osobowe, wyniki badań i historię pacjenta
Agent	Zarządzanie wizytami uwierzytelnionego pacjenta [M]	Agent ma dostęp do listy wizyt pacjenta i może nimi zarządzać

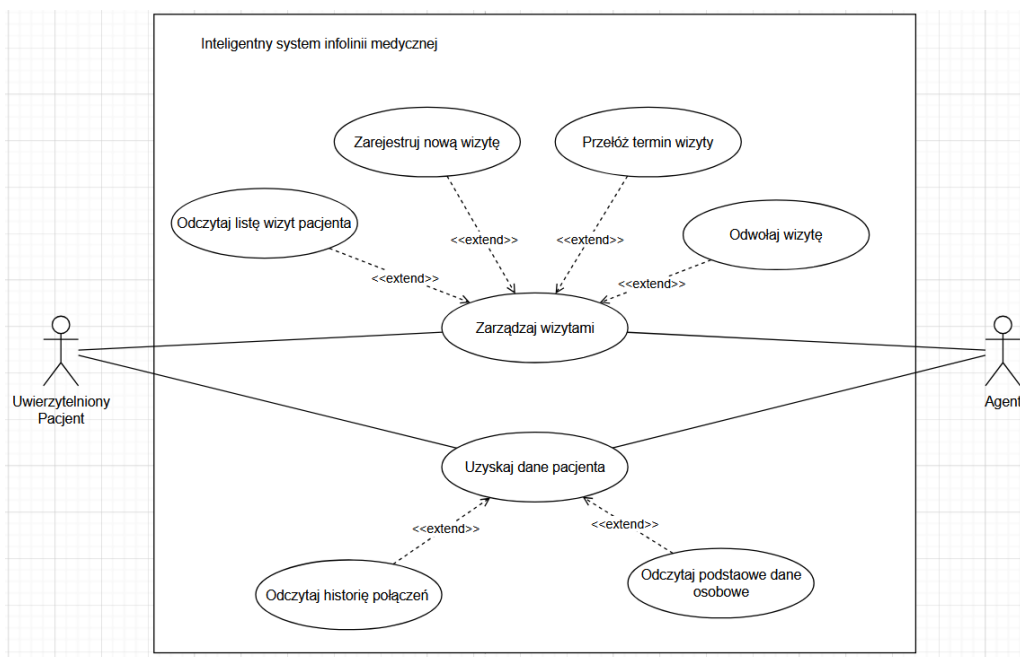
Tabela 1.8: Wymagania dotyczące obsługi przez agenta [źródło: opracowanie własne].

Aktor	Akcja	Wynik
Agent	Odebranie połączenie [M]	Agent odbiera otrzymane połączenie wraz z kontekstem dotychczasowej rozmowy
	Przetransferowanie połączenia do innego wyspecjalizowanego agenta [S]	Agent wykonuje połączenie do innego agenta lub na wybraną kolejkę w celu konsultacji lub przekierowania pacjenta do odpowiedniego obszaru obsługi
	Wyświetlenie danych pacjenta [M]	Agent posiada informacje potrzebne do identyfikacji i efektywnej obsługi pacjenta

Dodatkowo wybrane możliwe akcje przedstawione są na diagramach przypadków użycia, które szczegółowo ilustrują zależności między aktorami, a dostępnymi funkcjami. **Rysunek 1.1** określa podstawowe działania systemu, natomiast **Rysunek 1.2** prezentuje czynności osiągalne dla zautoryzowanych użytkowników związane z zarządzaniem danymi i wizytami.



Rysunek 1.1: Diagram przypadków użycia podstawowych działań [źródło: opracowanie własne].



Rysunek 1.2: Diagram przypadków użycia operacji po uwierzytelnieniu [źródło: opracowanie własne].

Mając jasno określone wymagania funkcjonalne, które stanowią punkt wyjścia do dalszego projektowania systemu, kolejnym etapem jest przejście do analizy wymagań poza funkcjonalnych.

2.5.2 Wymagania poza funkcjonalne

W tej części zawarte są wymagania poza funkcjonalne, które system CCaaS musi spełniać, aby zapewnić poprawne, stabilne i nieprzerwane działanie. Mając na uwadze kontekst środowiska medycznego, wymagania te mają szczególne znaczenie, ponieważ ich niespełnienie może wpływać na bezpieczeństwo i zdrowie pacjentów lub narazić placówkę na poważne konsekwencje prawne.

Tabela 1.9: Wymagania poza funkcjonalne rozważanego systemu CCaaS [źródło: opracowanie własne].

Obszar wymagań	Nr.	Opis
Użytkowanie	1.	System obsługuje język polski i angielski oraz poprawnie interpretuje potoczne sformułowania i regionalne gwary.

Obszar wymagań	Nr.	Opis
	2.	System generuje komunikaty zrozumiałe dla osób bez specjalistycznej wiedzy medycznej.
	3.	System obsługuje połączenia przychodzące z tradycyjnych sieci PSTN oraz sieci komórkowych.
	4.	Interfejs agenta jest czytelny, spójny i prezentuje wyłącznie dostępne funkcje zgodnie z rolą użytkownika.
Bezpieczeństwo	5.	Dane pacjentów, wrażliwe logi, nagrania rozmów są szyfrowane w spoczynku (AES-256) i tranzyście (TLS 1.2+).
	6.	Dostęp do systemu jest ściśle kontrolowany przez AWS IAM według zasady najmniejszych przywilejów.
	7.	System spełnia wymogi RODO oraz standardu HIPAA .w zakresie przetwarzania danych pacjentów.
Dostępność	8.	Automatyczna część infolinii jest dostępna dla pacjentów bez przerw 24/7/365.
	9.	Dostępność systemu wynosi minimum 99,9%.
	10.	Zaplanowane przerwy techniczne nie mogą przekraczać 30 minut.
Wydajność	11.	System może obsłużyć co najmniej 1000 równoległych połączeń.
	12.	Latencja odpowiedzi nie może przekraczać 2 sekund dla 95. percentyla (p95).
	13.	Wszystkie komponenty systemu skalują się automatycznie.
	14.	Wskaźnik skuteczności rozpoznawania intencji wynosi co najmniej 85% na zbiorze testowym.
Utrzymanie i wsparcie	15.	Infrastruktura systemu jest zdefiniowana jako kod (IaC) przy użyciu AWS CDK.
	16.	Wdrażanie aktualizacji systemu odbywa się w pełni przez potok CI/CD umożliwiając obsługę połączeń bez przerw.
	17.	Wszystkie dane pacjentów, logi, nagrania rozmów są przechowywane przez okres wymagany przepisami prawa, a następnie automatycznie usuwane.

Przeprowadzona analiza i ustanowione wymagania stanowią podstawę do dalszego projektowania i implementacji rozważanego systemu CCaaS. W kolejnym rozdziale przedstawiona została architektura proponowanego rozwiązania wraz z jego głównymi komponentami i przyjętymi założeniami.

3 PROJEKTOWANIE INTELIGENTNEGO SYSTEMU CCAAS

Niniejszy rozdział obejmuje kompleksowe projektowanie inteligentnego systemu infolinii medycznej zbudowanego w oparciu o platformę Amazon Connect Customer oraz wybranych usług ekosystemu AWS. Projektowanie zostało podzielone na trzy główne obszary: opracowanie koncepcji systemu wraz z definicją wariantów obsługi, projektowanie logiki przepływu połączeń w postaci diagramów przepływów i drzewa decyzyjnego bota, a następnie projekt architektury systemowej z technicznym opisem przepływu sekwencji pomiędzy głównymi komponentami.

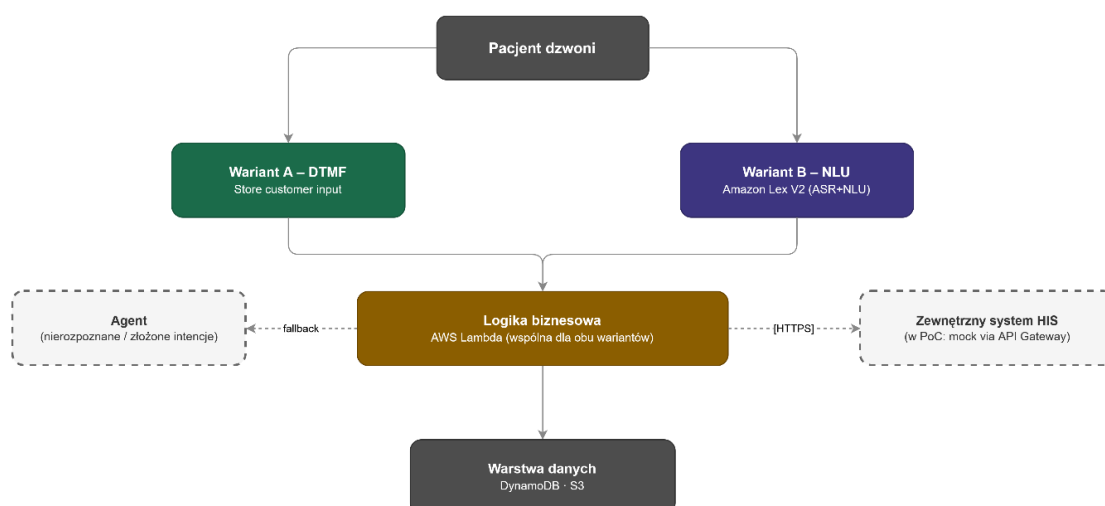
3.1 Opracowanie koncepcji inteligentnego systemu infolinii medycznej

Koncepcja systemu opiera się na hybrydowym modelu obsługi połączeń przychodzących, który łączy dwa komplementarne warianty interakcji z pacjentem. Pierwszym jest klasyczny tryb DTMF (ang. Dual-Tone Multi-Frequency), w którym pacjent nawiguje przez menu tonowe za pomocą klawiszy telefonu. Drugim jest zaawansowany tryb NLU (ang. Natural Language Understanding), w którym pacjent prowadzi swobodny dialog głosowy z botem konwersacyjnym opartym na Amazon Lex V2. Oba warianty zostają zintegrowane z tą samą warstwą logiki biznesowej realizowanej przez AWS Lambda oraz z centralnym systemem danych medycznych dostępnym przez Amazon API Gateway.

Głównym celem projektowanego systemu jest automatyczna obsługa powtarzalnych zgłoszeń pacjentów bez angażowania konsultanta: rejestracja i odwoływanie wizyt, uzyskanie informacji o godzinach otwarcia placówki oraz dostęp do podstawowych danych pacjenta po uprzednim uwierzytelnieniu. W przypadku złożonych lub nierozpoznanych intencji system automatycznie przekierowuje połączenie do kolejki agentów, zachowując pełen kontekst dotychczasowej rozmowy, jako atrybut kontaktu Amazon Connect.

System jest zaprojektowany z myślą o obsłudze błędów. W przypadku, gdy niezbędny komponent jest niedostępny, co uniemożliwia standardowy przepływ połączenia, połączenie zostaje natychmiast przekierowane do agenta bez przerywania rozmowy. Architektura jest w pełni bezstanowa po stronie Amazon Connect. Cały stan sesji przechowywany jest w atrybutach kontaktu lub tymczasowo w Amazon DynamoDB, co umożliwia skalowanie do tysięcy równoległych połączeń bez dodatkowej konfiguracji infrastruktury.

Wariant A (DTMF) charakteryzuje się niską złożonością implementacyjną i pełną przewidywalnością ścieżek dialogowych. Każda opcja jest jawnie zdefiniowana w Contact Flow, co ułatwia testowanie i utrzymanie. Wadą jest ograniczona elastyczność: pacjent musi stosować się do sztywnej struktury menu i poruszać się zgodnie z predefiniowaną hierarchią opcji. Wariant B (NLU) odwraca tę zależność – system dopasowuje się do pacjenta, a nie odwrotnie. Dzięki Amazon Lex V2 wystarczy naturalna wypowiedź, by system rozpoznał intencję i zebrał wymagane sloty. Ceną za tę elastyczność jest wyższa złożoność konfiguracji (modele językowe, walidacja slotów, obsługa fallbacków) oraz potencjalnie większa latencja wynikająca z przetwarzania ASR i NLU. **Rysunek 3.1** ilustruje koncepcję hybrydowego modelu pokazując, że niezależnie od wybranej ścieżki wejścia oba warianty współdzielą tę samą warstwę logiki biznesowej i warstwę danych, co zapewnia spójność wyników.



Rysunek 3.1: Koncepcja hybrydowego modelu obsługi połączeń głosowych [źródło: opracowanie własne].

W projektowanym systemie wybór wariantu nie wpływa na wynik operacji – pacjent rejestrujący wizytę przez menu DTMF i pacjent robiący to samo głosem otrzymują identyczną odpowiedź, ponieważ obaj docierają do tej samej funkcji Lambda i tego samego systemu danych oraz centralnego systemu medycznego.

3.2 Projektowanie procesów obsługi połączeń

Projektowanie procesów obsługi połączeń obejmuje opracowanie diagramów czynności dla obu wariantów systemu oraz projekt drzewa decyzyjnego bota konwersacyjnego. Celem

jest precyzyjne odwzorowanie każdego etapu interakcji pacjenta z systemem: od momentu nawiązania połączenia, przez procesy uwierzytelnienia i realizacji intencji, aż do rozłączenia lub transferu do agenta.

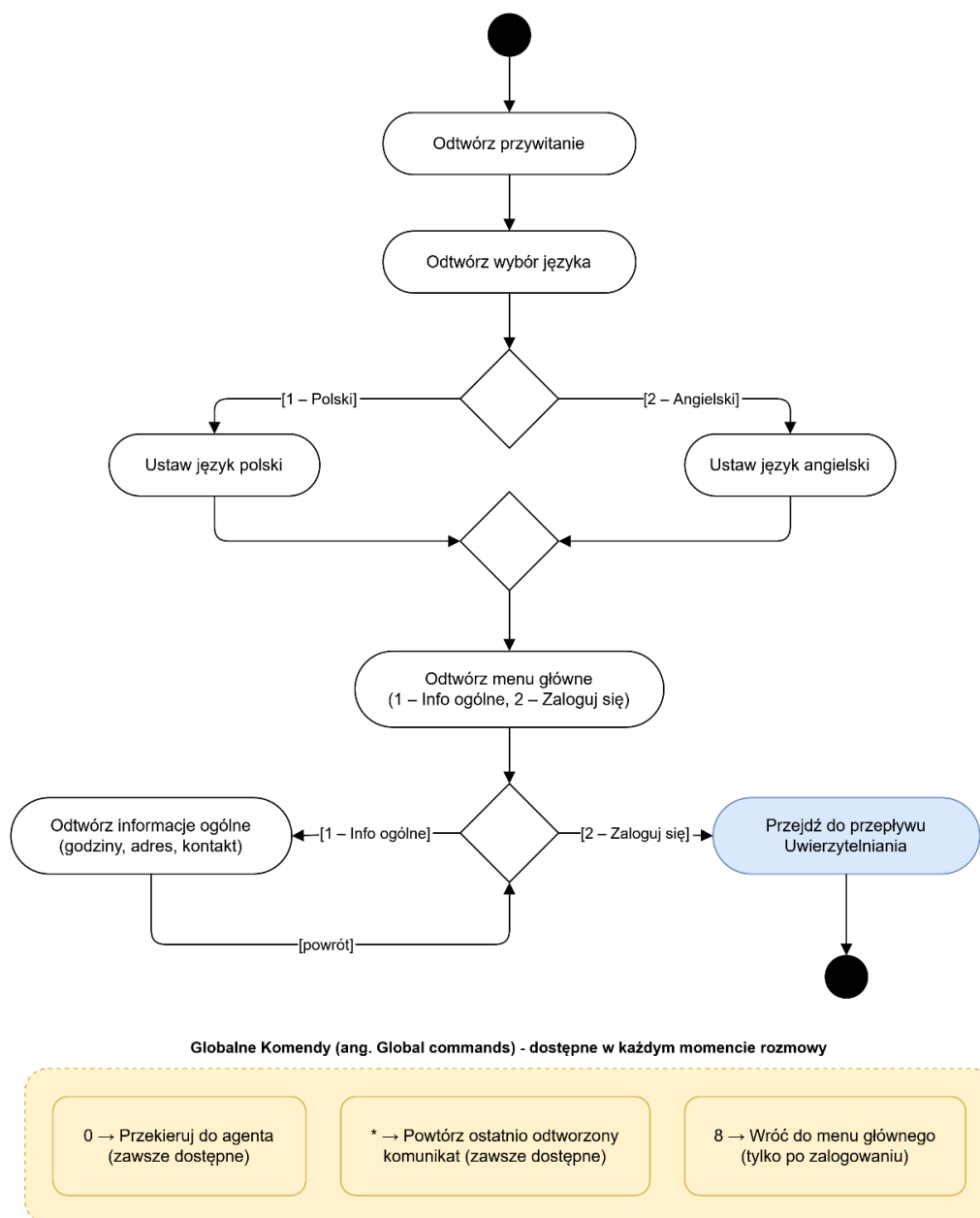
3.2.1 Wariant A – Przepływ IVR sterowany tonowym DTMF

Wariant A stanowi implementację klasycznego systemu IVR opartego na sygnalizacji DTMF. Po nawiązaniu połączenia system odtwarza powitalny komunikat głosowy i prezentuje pacjentowi menu główne. Pacjent nawiguje przez system wciskając odpowiednie klawisze numeryczne, a na podstawie odebranej cyfry przepływ rozgałęzia się do odpowiadającej opcji lub podmenu. W przypadku trzykrotnego błędu wyboru lub upływu czasu oczekiwania na wejście system automatycznie przekierowuje połączenie do kolejki agentów. Menu główne oferuje pacjentowi następujące opcje:

- Informacje ogólne (godziny otwarcia, adres, dostępne usługi).
- Zaloguj się (wymaga podania numeru PESEL, numeru telefonu i kodu OTP).
- Połączenie z konsultantem.

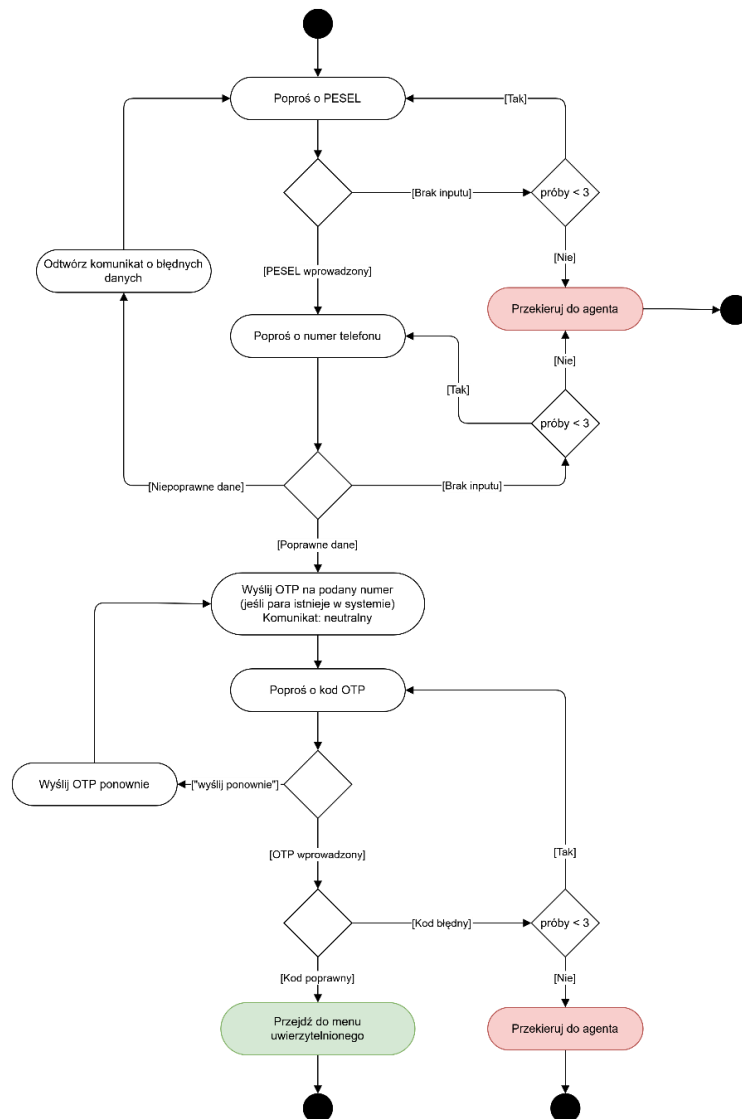
Po pomyślnym uwierzytelnieniu pacjent uzyskuje dostęp do menu chronionego, obejmującego rejestrację wizyty, przeglądanie i zarządzanie zarezerwowanymi terminami oraz wgląd w dane osobowe zarejestrowane w systemie.

Rysunek 3.2 ilustruje inicjalizację połączenia i obsługę menu głównego w Wariancie A (DTMF). Po nawiązaniu połączenia system odtwarza przywitanie i umożliwia pacjentowi wybór języka obsługi poprzez wciśnięcie odpowiedniego klawisza. Następnie prezentowane jest menu główne oferujące dostęp do informacji ogólnych lub logowania do strefy pacjenta. Diagram uwzględnia również zestaw globalnych komend dostępnych w każdym momencie rozmowy, takich jak transfer do agenta czy powrót do menu głównego.



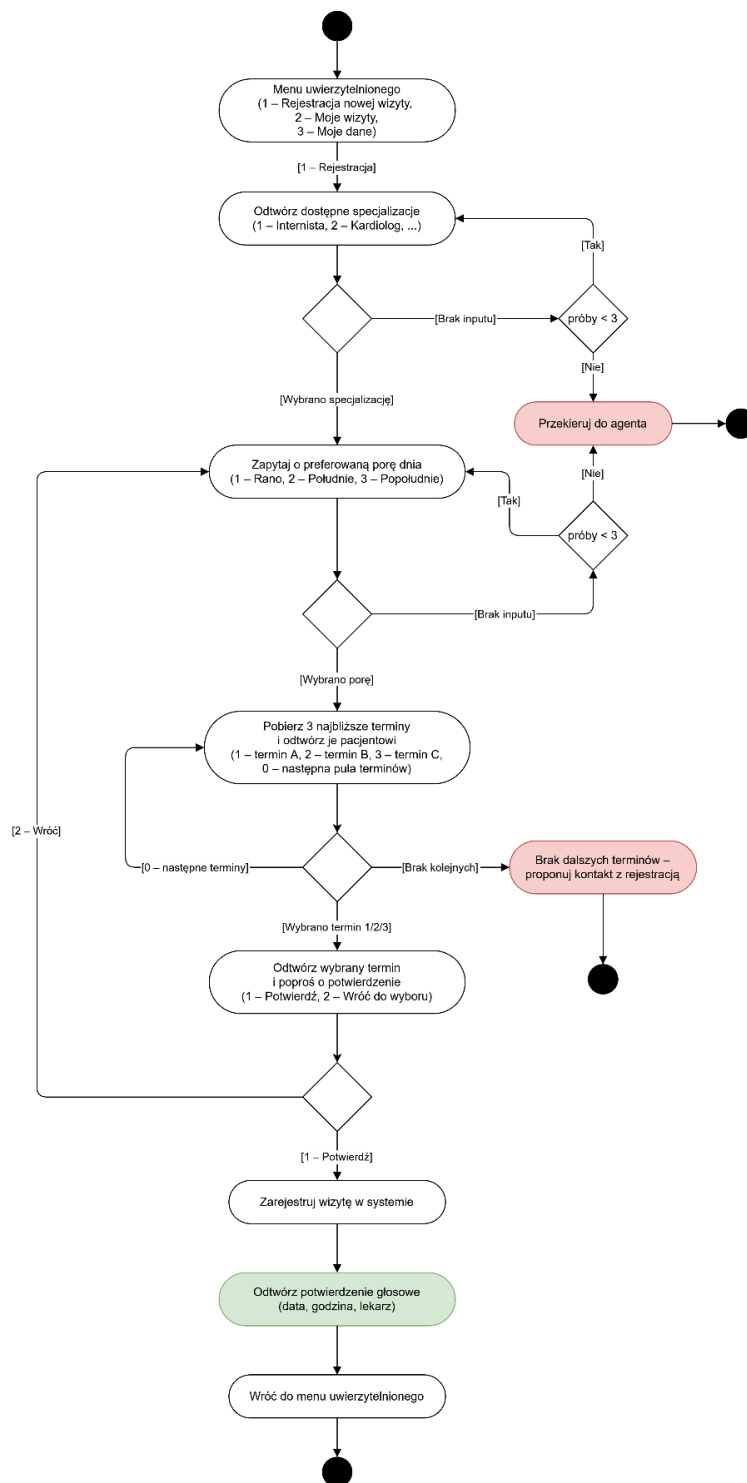
Rysunek 3.2: Diagram czynności – inicjalizacja połączenia i menu główne (Wariant A, DTMF) - źródło: opracowanie własne.

Rysunek 3.3 dotyczy procesu uwierzytelniania pacjenta w Wariacie A (DTMF). Weryfikacja tożsamości opiera się na dwóch składnikach: numerze PESEL oraz numerze telefonu, które system sprawdza jako parę. Komunikat o wysłaniu kodu OTP jest celowo neutralny i nie ujawnia czy podane dane istnieją w systemie. Po poprawnym wprowadzeniu kodu jednorazowego pacjent uzyskuje dostęp do funkcji wymagających uwierzytelnienia. Każdy etap przewiduje maksymalnie trzy próby, po których następuje automatyczne przekierowanie do agenta.



Rysunek 3.3: Diagram czynności – uwierzytelnianie pacjenta (Wariant A, DTMF) - źródło: opracowanie własne.

Rysunek 3.4 obejmuje proces rejestracji wizyty w Wariacie A (DTMF). Po uwierzytelnieniu pacjent wybiera specjalizację medyczną oraz preferowaną porę dnia, na podstawie których system pobiera i odtwarza trzy najbliższe dostępne terminy. Wyboru dokonuje się poprzez wciśnięcie odpowiedniej cyfry, z możliwością przejścia do kolejnej puli terminów. Po wyborze system odtwarza szczegóły wizyty i prosi o potwierdzenie, po którym wizyta zostaje zarejestrowana w systemie.



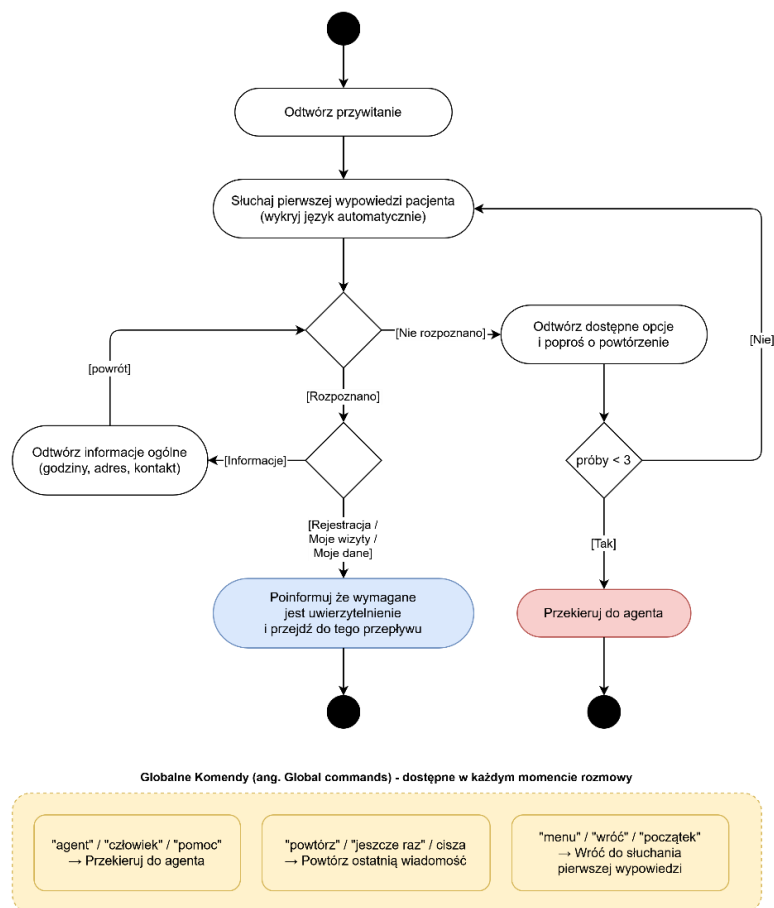
Rysunek 3.4: Diagram czynności – rejestracja wizyty (Wariant A, DTMF) - źródło: opracowanie własne.

3.2.2 Wariant B – Przepływ IVR sterowany głosowym NLU

Wariant B realizuje obsługę połączenia za pośrednictwem bota konwersacyjnego Amazon Lex V2. Po nawiązaniu połączenia system odtwarza krótkie powitanie i natychmiast przechodzi do nasłuchu – język obsługi wykrywany jest automatycznie na podstawie pierwszej wypowiedzi pacjenta, bez konieczności jawnego wyboru. Pacjent może swobodnie wyrazić swoją potrzebę, a bot rozpoznaje intencję i – jeśli wypowiedź była niepełna – dopytuje jedynie o brakujące informacje. Przykładowo wypowiedź „chcę umówić się do kardiologa na przyszły tydzień rano” jest wystarczająca, by system przeszedł bezpośrednio do prezentacji dostępnych terminów bez zadawania kolejnych pytań. W przypadku trzykrotnego braku rozpoznania intencji system automatycznie przekierowuje połączenie do agenta.

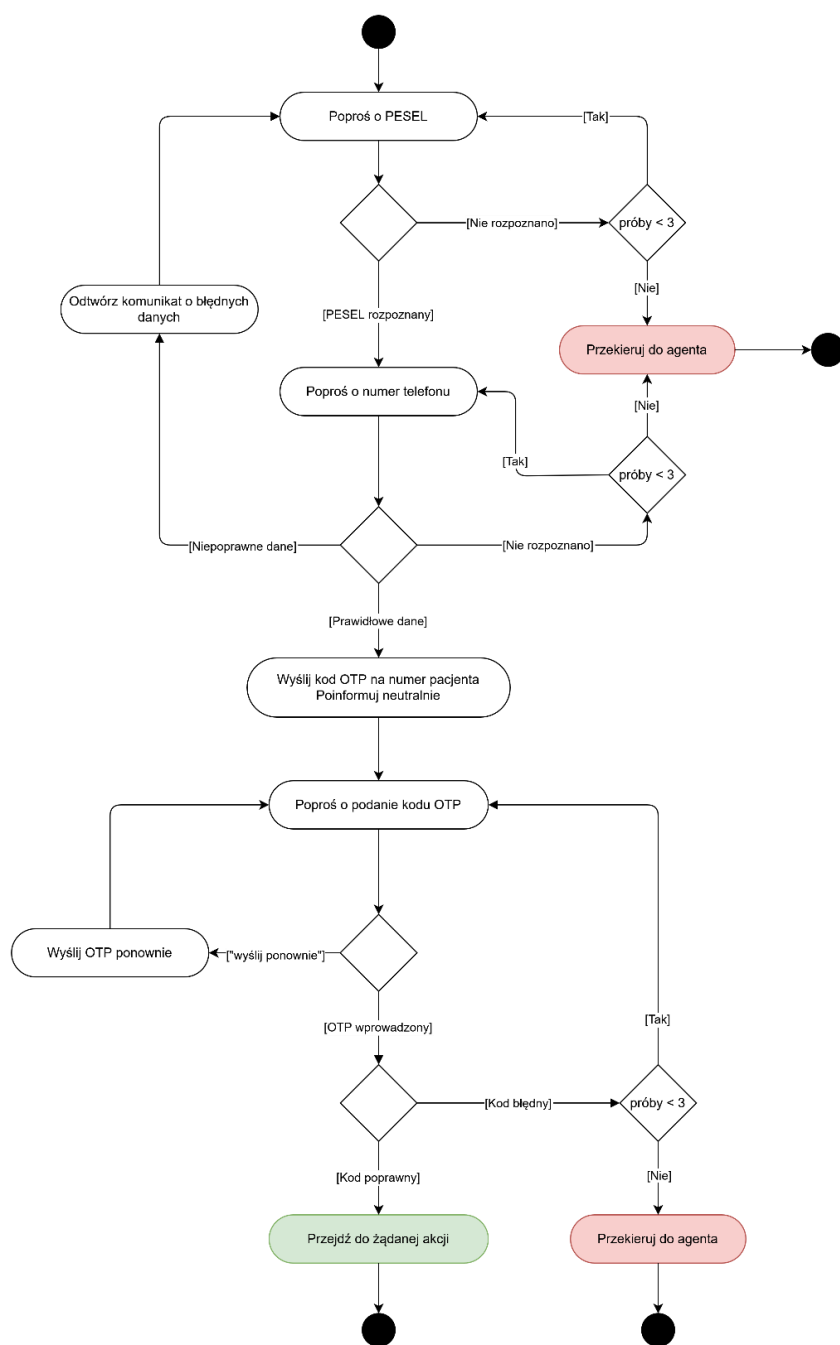
Kluczową różnicą w stosunku do Wariantu A jest brak sztywnej struktury menu – pacjent nie musi znać dostępnych opcji ani wcisnąć konkretnego klawisza. Akcje wymagające uwierzytelnienia, takie jak rejestracja wizyty czy dostęp do danych pacjenta, inicjują proces weryfikacji tożsamości automatycznie w momencie gdy bot rozpozna stosowną intencję, bez odrębnego kroku logowania w menu.

Rysunek 3.5 przedstawia inicjalizację połączenia i rozpoznanie intencji w Wariacie B (NLU). W odróżnieniu od Wariantu A język obsługi wykrywany jest automatycznie na podstawie pierwszej wypowiedzi pacjenta, bez konieczności jawnego wyboru. System nasłuchuje swobodnej wypowiedzi i klasyfikuje intencję – dostęp do informacji ogólnych lub akcji wymagających uwierzytelnienia inicjowany jest bezpośrednio z wypowiedzi, bez nawigacji po menu. Uwzględniono globalnie dostępne komendy głosowe oraz mechanizm fallback przekierowujący do agenta po trzykrotnym braku rozpoznania intencji.



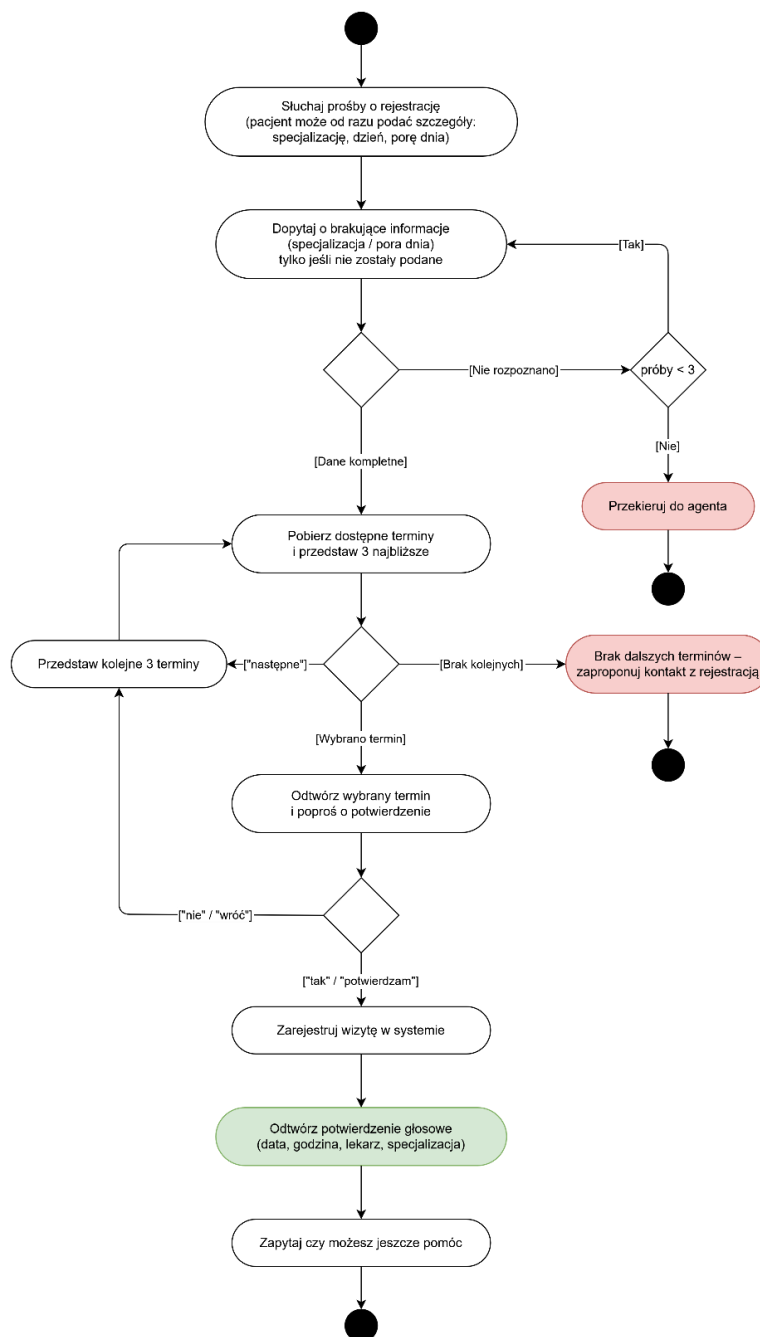
Rysunek 3.5: Diagram czynności – inicjalizacja połączenia i rozpoznanie intencji (Wariant B, NLU) - źródło: opracowanie własne.

Rysunek 3.6 odnosi się do uwierzytelniania pacjenta w Wariancie B (NLU). Przebieg jest analogiczny do Wariantu A – weryfikacja pary PESEL i numer telefonu, wysyłka kodu OTP z neutralnym komunikatem oraz weryfikacja kodu jednorazowego – z tą różnicą że dane zbierane są przez rozpoznawanie mowy zamiast klawiatury DTMF. Przewidziano możliwość ponownego wysłania kodu OTP na żądanie głosowe pacjenta oraz automatyczne przekierowanie do agenta po trzykrotnym nieudany wprowadzeniu danych na każdym etapie.



Rysunek 3.6: Diagram czynności – uwierzytelnianie pacjenta (Wariant B, NLU) - źródło: opracowanie własne.

Rysunek 3.7 dotyczy rejestracji wizyty w Wariancie B (NLU). Kluczową różnicą względem Wariantu A jest możliwość podania wszystkich wymaganych informacji w jednej swobodnej wypowiedzi – pacjent może od razu określić specjalizację i preferowaną porę dnia, a system dopytuje jedynie o brakujące dane. Wybór terminu realizowany jest przez mowę naturalną, a potwierdzenie rejestracji przez odpowiedź twierdzącą lub przeczącą. Po zakończeniu system pyta czy może jeszcze pomóc, pozostając w trybie nasłuchu.



Rysunek 3.7: Diagram czynności – rejestracja wizyty (Wariant B, NLU) - źródło: opracowanie własne.

Przedstawione diagramy czynności wyczerpują opis przepływów sterowania dla obu wariantów systemu stanowiąc kompletną specyfikację na poziomie interakcji z pacjentem.

3.2.3 Projekt drzewa decyzyjnego bota konwersacyjnego

Bot konwersacyjny Amazon Lex V2 został zaprojektowany w oparciu o hierarchiczne drzewo intencji podzielone na trzy warstwy dostępu. Pierwszą stanowią intencje globalne, dostępne w każdym momencie rozmowy niezależnie od stanu sesji – należą do nich obsługa menu głównego, udostępnianie informacji ogólnych o placówce, powtarzanie ostatniego komunikatu oraz transfer do agenta. Drugą warstwę tworzą intencje składające się na sam proces uwierzytelniania, czyli zbieranie i weryfikację pary PESEL i numeru telefonu, obsługę kodu OTP oraz jego ponowne wysłanie na żądanie pacjenta. Trzecią warstwę stanowią intencje wymagające uwierzytelnienia, dostępne wyłącznie po pomyślnej weryfikacji tożsamości – obejmują one rejestrację, przeglądanie, przełożenie i odwołanie wizyt oraz dostęp do danych pacjenta.

Każda intencja definiuje nazwę, zestaw wzorców wypowiedzi inicjujących rozpoznanie, opcjonalne sloty z walidacją danych wejściowych oraz realizowaną funkcję biznesową. Sloty pozwalają botowi na zbieranie ustrukturyzowanych informacji z wypowiedzi pacjenta – takich jak specjalizacja medyczna, preferowana pora dnia czy identyfikator wizyty – i dopytywanie o brakujące dane bez konieczności ponownego inicjowania intencji. W przypadku trzykrotnego braku rozpoznania intencji lub przekroczenia liczby dozwolonych prób wprowadzenia danych system automatycznie przekazuje połączenie do agenta.

Tabela 3.1 opisuje specyfikację intencji bota konwersacyjnego Amazon Lex V2, obejmującą przykładowe wypowiedzi inicjujące, definiowane sloty oraz odpowiadające operacje biznesowe.

Tabela 3.1: Specyfikacja intencji bota konwersacyjnego Amazon Lex V2 [źródło: opracowanie własne].

Intencja	Warstwa	Przykładowe wypowiedzi	Sloty	Operacja biznesowa
Warstwa globalna – dostępne w każdym momencie rozmowy				
MainMenuIntent	Globalna	„dzień dobry”, „co mogę zrobić”, „menu”	–	Rozpoznanie intencji i przekierowanie do odpowiedniego przepływu
InfoIntent	Globalna	„jaki są godziny otwarcia”, „gdzie się znajduje”, „podaj adres”	–	Odtworzenie informacji ogólnych o placówce

Intencja	Warstwa	Przykładowe wypowiedzi	Sloty	Operacja biznesowa
RepeatIntent	Globalna	„powtórz”, „słucham?”, „nie dosłyszałem”	–	Powtórzenie ostatniego komunikatu systemu
AgentTransferIntent	Globalna	„połącz z agentem”, „chcę rozmawiać z człowiekiem”, „pomoc”	–	Transfer połączenia do agenta
FallbackIntent	Globalna	(nie rozpoznano intencji)	–	Dopytanie o intencję lub transfer do agenta po trzeciej nieudanej próbie
Warstwa uwierzytelniania – składają się na proces weryfikacji tożsamości				
AuthIntent	Uwierzytelnianie	„chcę się zalogować”, (każda chroniona akcja)	PESEL, Telefon	Weryfikacja tożsamości pacjenta na podstawie pary PESEL i numer telefonu
OtpIntent	Uwierzytelnianie	(po wysłaniu OTP)	KodOTP	Weryfikacja kodu jednorazowego
ResendOtpIntent	Uwierzytelnianie	„wyślij ponownie”, „nie dostałem kodu”	–	Ponowne wysłanie kodu OTP na numer pacjenta
Warstwa wymagająca uwierzytelnienia – dostępne po pomyślnej weryfikacji tożsamości				
PatientDataIntent	Wymagająca uwierzytelnienia	„moje dane”, „pokaż moje dane”	–	Pobranie i odtworzenie danych pacjenta z systemu
AppointmentsIntent	Wymagająca uwierzytelnienia	„moje wizyty”, „jakie mam terminy”	–	Pobranie i odtworzenie listy zarezerwowanych wizyt
BookingIntent	Wymagająca uwierzytelnienia	„chcę umówić wizytę”, „zarejestruj mnie do kardiologa”,	Specjalizacja, PoraDnia, WybranyTermin	Pobranie dostępnych terminów i

Intencja	Warstwa	Przykładowe wypowiedzi	Sloty	Operacja biznesowa
		„umów mnie na rano"		rejestracja wybranej wizyty
NextSlotsIntent	Wymagająca uwierzytelnienia	„następne", „pokaż więcej", „inne opcje"	–	Pobranie kolejnej puli dostępnych terminów
RescheduleIntent	Wymagająca uwierzytelnienia	„chcę przełożyć wizytę", „zmień termin"	WizytaID, NowaPoraDnia, NowyTermin	Pobranie nowych terminów i aktualizacja istniejącej rezerwacji
CancelIntent	Wymagająca uwierzytelnienia	„odwołaj wizytę", „anuluj moją wizytę"	WizytaID	Odwołanie wskazanej wizyty w systemie
ConfirmationIntent	Wymagająca uwierzytelnienia	„tak", „potwierdzam", „zgadza się"	–	Zatwierdzenie akcji w bieżącym kontekście sesji
DenyIntent	Wymagająca uwierzytelnienia	„nie", „wróć", „zmień"	–	Anulowanie akcji i powrót do poprzedniego kroku

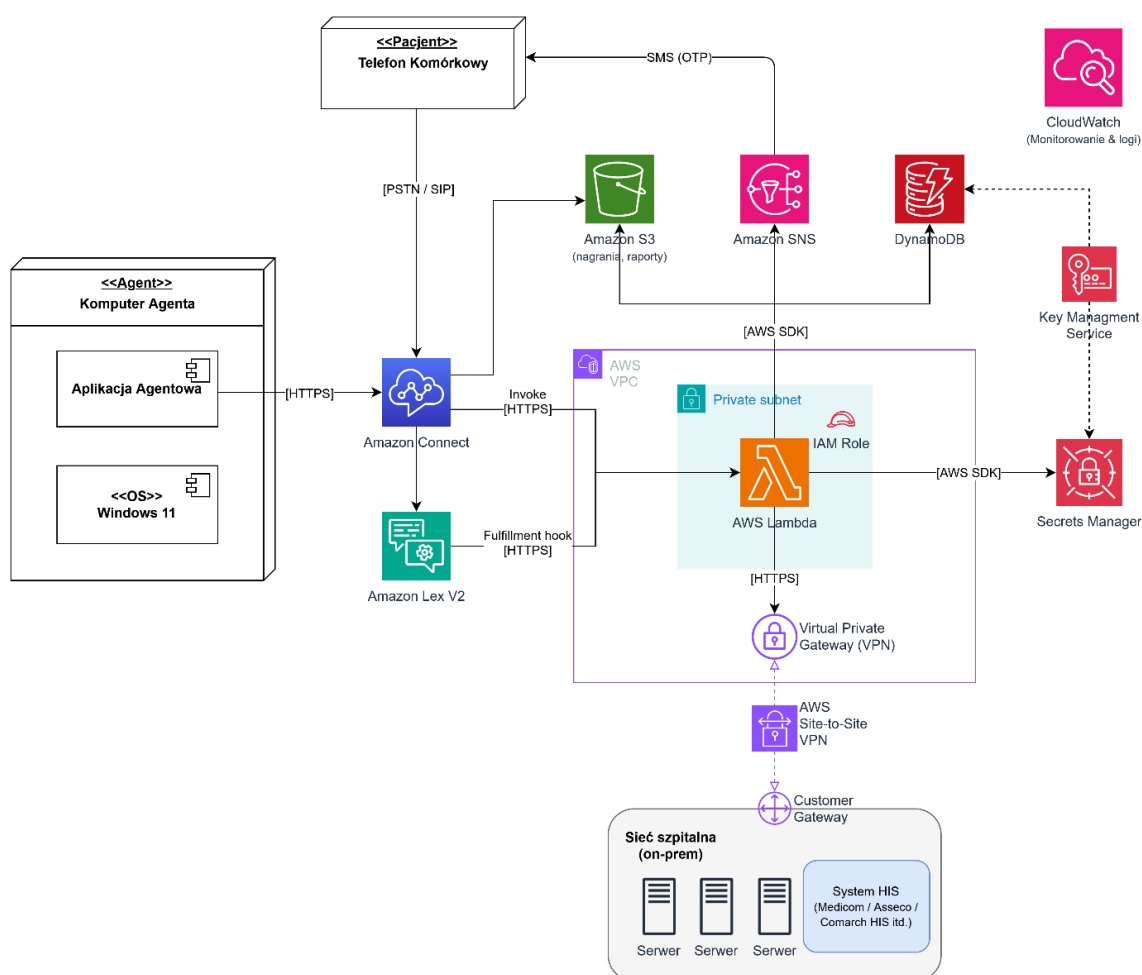
Mając zdefiniowane intencje i przepływy dialogowe obu wariantów systemu, kolejnym krokiem jest zaprojektowanie architektury technicznej, która będzie je realizować.

3.3 Projekt architektury inteligentnego systemu infolinii medycznej

Architektura projektowanego systemu opiera się na modelu event-driven, w którym każde zdarzenie (np. nawiązanie połączenia, wciśnięcie klawisza, wypowiedź pacjenta) wyzwala odpowiedni łańcuch przetwarzania bez konieczności utrzymywania stale działających serwerów. Wszystkie komponenty są w pełni zarządzane przez AWS i automatycznie skalowane do rzeczywistych potrzeb.

3.3.1 Diagram architektury systemowej

Rysunek 3.8 przedstawia docelową architekturę systemu automatycznej obsługi połączeń głosowych. Centralnym elementem jest Amazon Connect, który odbiera połączenia przychodzące od pacjentów i kieruje sterowanie bezpośrednio do AWS Lambda (Wariant A, DTMF) lub przez Amazon Lex V2 (Wariant B, NLU). Lambda realizuje całą logikę biznesową systemu, komunikując się z Amazon DynamoDB w zakresie sesji OTP i logów wywołań, Amazon SNS w celu wysyłki kodów jednorazowych oraz AWS Secrets Manager w celu pobierania poświadczeń do systemów zewnętrznych. Nagrania rozmów i dane raportowe składowane są w Amazon S3, a całość środowiska objęta jest monitoringiem przez Amazon CloudWatch. Komunikacja z zewnętrznym systemem HIS realizowana jest docelowo przez szyfrowany tunel AWS Site-to-Site VPN. W środowisku PoC połączenie to zastąpione jest przez mock oparty na Amazon API Gateway.



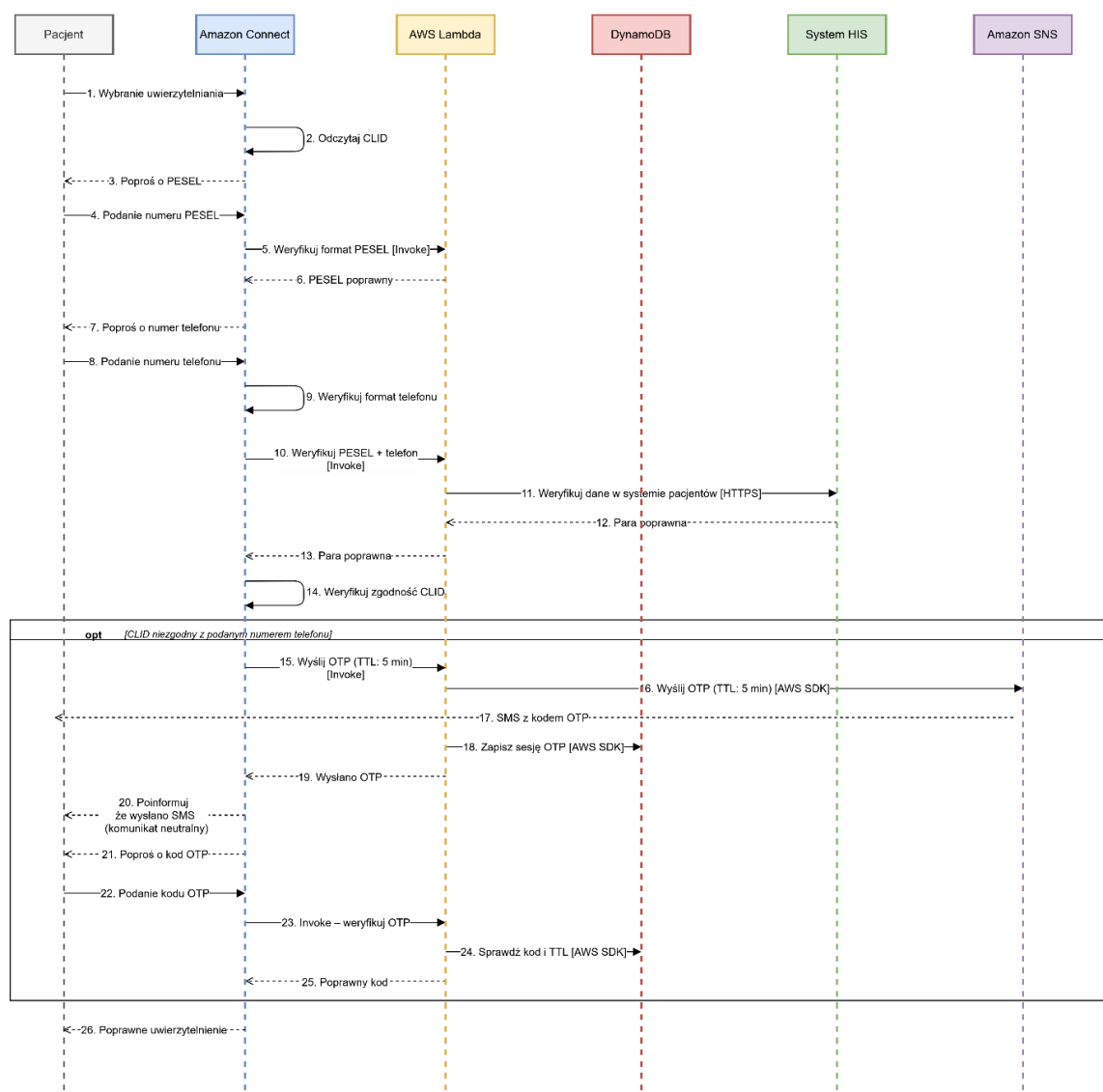
Rysunek 3.8: Diagram Architektury systemu – widok komponentów i integracji [źródło: opracowanie własne].

Architektura systemu określa jakie komponenty wchodzi w skład rozwiązania i jak są ze sobą powiązane. W kolejnym podrozdziale omówiony zostanie również istotny aspekt ich współpracy.

3.3.2 Przepływ zdarzeń między Connect, Lex i Lambda

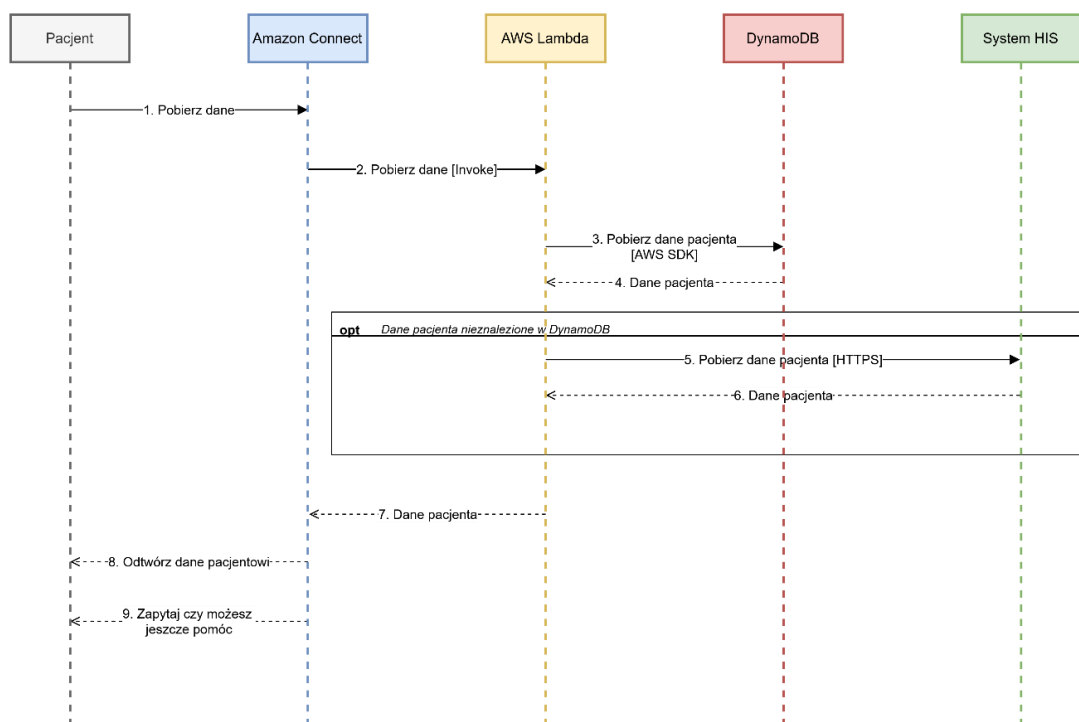
Sercem architektury jest model zdarzeniowy, w którym każde działanie pacjenta lub systemu wywołuje ściśle określone zdarzenie i następny krok przetwarzania. Niezależnie od wariantu interakcji – DTMF lub NLU – przepływ sterowania przebiega przez te same warstwy: warstwę obsługi połączenia (Amazon Connect), opcjonalną warstwę rozumienia języka (Amazon Lex V2) oraz warstwę logiki biznesowej (AWS Lambda). Różnica między wariantami sprowadza się wyłącznie do sposobu zbierania danych od pacjenta (przez klawiaturę lub mowę), natomiast logika biznesowa i warstwa danych pozostają identyczne dla obu przypadków.

Rysunek 3.9 przedstawia proces uwierzytelniania pacjenta, obejmujący weryfikację pary PESEL i numer telefonu, wysyłkę kodu OTP oraz jego walidację przed udzieleniem dostępu do chronionych funkcji systemu.



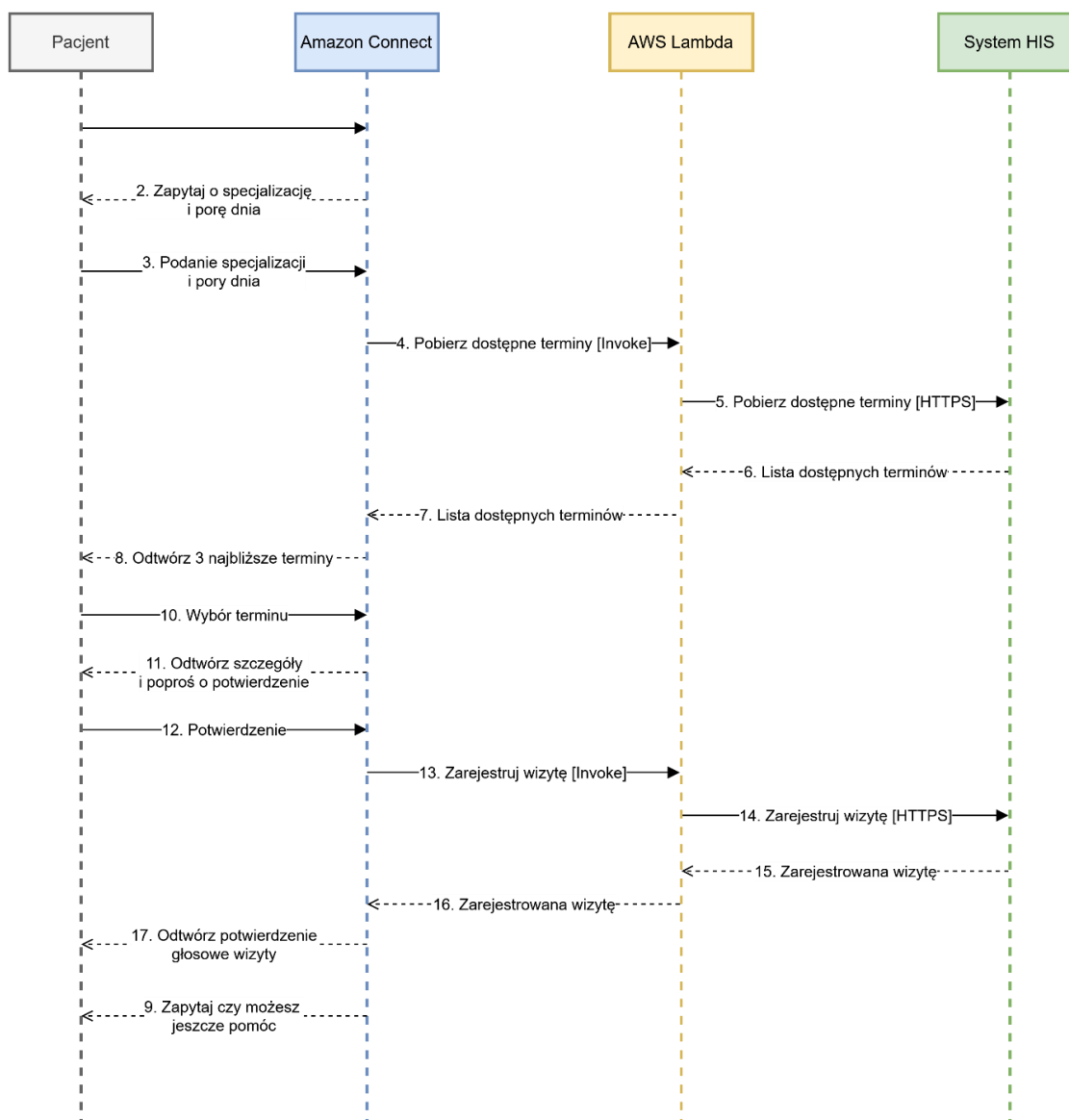
Rysunek 3.9: Diagram sekwencji – uwierzytelnianie pacjenta [źródło: opracowanie własne].

Rysunek 3.10 ukazuje pobieranie danych pacjenta i informacji o placówce, pokazując w jaki sposób system odpytuje warstwę danych i zwraca wynik do pacjenta w trakcie trwania połączenia.



Rysunek 3.10: Diagram sekwencji – pobieranie danych pacjenta [źródło: opracowanie własne].

Rysunek 3.11 prezentuje sekwencję zdarzeń dla scenariusza rejestracji wizyty: od określenia intencji pacjenta, pobranie dostępnych terminów i potwierdzenie rezerwacji, aż do zapisu w systemie.



Rysunek 3.11: Diagram sekwencji – rejestracja nowej wizyty [źródło: opracowanie własne].

Przedstawione diagramy sekwencji potwierdzają że przyjęty model zdarzeniowy zapewnia spójność przepływu sterowania niezależnie od złożoności scenariusza, a separacja warstw umożliwia niezależne rozwijanie logiki biznesowej bez ingerencji w konfigurację platformy telefonicznej.

3.3.3 Integracje komponentów i model bezpieczeństwa

Wszystkie integracje między komponentami realizowane są przez mechanizmy natywne dla ekosystemu AWS, bez potrzeby stosowania zewnętrznych brokerów komunikatów. Amazon Connect wywołuje AWS Lambda synchronicznie (timeout 8s) lub

asynchronicznie (timeout 60s) bezpośrednio z Contact Flow. Amazon Lex V2 wywołuje Lambda jako fulfillment hook dla każdej intencji, przekazując przetworzone sloty i metadane sesji w formacie JSON. Lambda komunikuje się z zewnętrznym systemem medycznym przez Amazon API Gateway REST API, stosując uwierzytelnienie przez klucze IAM lub tokeny JWT.

Model bezpieczeństwa opiera się na zasadzie minimalnych uprawnień (ang. least privilege). Każda funkcja Lambda posiada dedykowaną rolę IAM z dostępem wyłącznie do niezbędnych zasobów. Dane pacjentów przechowywane w DynamoDB i nagrania w S3 są szyfrowane kluczami AWS KMS (AES-256). Komunikacja między wszystkimi komponentami odbywa się wyłącznie przez szyfrowane kanały TLS 1.2+. Amazon CloudWatch gromadzi logi i metryki ze wszystkich warstw systemu, umożliwiając bieżący monitoring i automatyczne alerty przez Amazon SNS w przypadku przekroczenia zdefiniowanych progów (np. wzrost latencji Lambda powyżej 2000 ms, wzrost wskaźnika błędów powyżej 5%).

Wszystkie integracje między komponentami systemu realizowane są przez wewnętrzne API AWS, bez konieczności konfigurowania osobnych kanałów komunikacyjnych ani stosowania zewnętrznych brokerów komunikatów. Jest to bezpośrednia konsekwencja wyboru jednorodnego ekosystemu AWS i jeden z głównych argumentów przemawiających za tą platformą w stosunku do Twilio Flex.

Opracowana architektura jest w pełni zgodna z wymaganiami poza funkcjonalnymi zdefiniowanymi w podrozdziale 2.5.2. Automatyczna skalowalność wszystkich komponentów AWS zapewnia obsługę co najmniej 1000 równoległych połączeń bez ręcznej ingerencji. Model event-driven eliminuje stan trwały po stronie platformy, redukując latencję i upraszczając skalowanie. Wdrożenie całej infrastruktury jako kodu (AWS CDK) zapewnia powtarzalność, kontrolę wersji i możliwość błyskawicznego odtworzenia środowiska w przypadku awarii. Szczegółowy opis implementacji poszczególnych komponentów zostanie przedstawiony w kolejnym rozdziale.

Bibliografia

- [1] D. Papaj, „FaaS: co to jest i jak serverless computing zmienia tworzenie aplikacji,” 29 październik 2025. [Online]. Available: <https://codelivery.eu/blog/faas-co-to-jest-i-jak-dziala/>.
- [2] AWS, „AWS Cloud Development Kit,” [Online]. Available: <https://aws.amazon.com/cdk/>.
- [3] AWS, „Amazon Connect Customer,” [Online]. Available: <https://aws.amazon.com/products/connect/customer/>.
- [4] H. Weld, „A Survey of Joint Intent Detection and Slot Filling Models in Natural Language Understanding,” 23 grudzień 2022. [Online]. Available: <https://dl.acm.org/doi/10.1145/3547138>.
- [5] I. Shalyminov, „Data-Efficient Methods for Dialogue Systems,” grudzień 2020. [Online]. Available: <https://arxiv.org/pdf/2012.02929>.
- [6] Grand View Research, „Contact Center as a Service Market,” 2026.
- [7] B. Jeppesen, „IVR vs. Conversational AI: Which Delivers and Which Delays,” 5 sierpień 2025. [Online]. Available: <https://www.cmswire.com/contact-center/ivr-vs-conversational-ai-which-delivers-and-which-delays/>.
- [8] Sejm Rzeczypospolitej Polskiej, „Ustawa o systemie informacji w ochronie zdrowia (Dz.U.2026.0.208 t.j.),” 28 kwiecień 2011. [Online]. Available: <https://lexlege.pl/ustawa-o-systemie-informacji-w-ochronie-zdrowia/rozdzial-1-przepisy-ogolne/6758/>.
- [9] Parlament Europejski i Rada Unii Europejskiej, „Rozporządzenie Parlamentu Europejskiego i Rady (UE) 2016/679 (RODO), Dz.Urz. UE L 119/1,” 27 kwiecień 2016. [Online]. Available: <https://eur-lex.europa.eu/legal-content/PL/TXT/?uri=CELEX:32016R0679>.
- [10] Parlament Europejski i Rada Unii Europejskiej, „Dyrektywa Parlamentu Europejskiego i Rady (UE) 2022/2555 w sprawie środków na rzecz wysokiego wspólnego poziomu cyberbezpieczeństwa na terytorium Unii (NIS2) Dz.Urz. UE L 333 z 27.12.2022, s. 80,” grudzień 14 2022. [Online].

- [11] U.S. Department of Health & Human Services, „Health Insurance Portability and Accountability Act of 1996 (HIPAA) Pub.L. 104–191, 110 Stat. 1936,” 21 sierpień 1996. [Online]. Available: <https://www.hhs.gov/hipaa/index.html>.
- [12] HL7 International, „Fast Healthcare Interoperability Resources (FHIR), wersja R4,” 2019. [Online]. Available: <https://hl7.org/fhir/>.
- [13] AWS, „Amazon Connect Customer Pricing - Appendix,” [Online]. Available: <https://aws.amazon.com/products/connect/customer/pricing/appendix>.
- [14] AWS, „Compliance in the cloud,” [Online]. Available: <https://aws.amazon.com/health/healthcare-compliance/?nc=sn&loc=4&dn=1>.
- [15] Twilio Flex, „Getting Started with TaskRouter,” [Online]. Available: <https://www.twilio.com/docs/flex/developer/routing>.
- [16] Twilio Flex, „Twilio Flex Pricing,” [Online]. Available: <https://www.twilio.com/en-us/flex/pricing>.
- [17] AWS, „Flows in Connect Customer,” [Online]. Available: <https://docs.aws.amazon.com/connect/latest/adminguide/connect-contact-flows.html>.
- [18] AWS, „Connect Customer service quotas,” [Online]. Available: <https://docs.aws.amazon.com/connect/latest/adminguide/amazon-connect-service-limits.html>.
- [19] AWS, „AWS Product and Service Pricing,” [Online]. Available: <https://aws.amazon.com/pricing/>.

Spis rysunków

Rysunek 2.1: Diagram przypadków użycia podstawowych działań.	27
Rysunek 2.2: Diagram przypadków użycia operacji po uwierzytelnieniu.	28
Rysunek 3.1: Koncepcja hybrydowego modelu obsługi połączeń głosowych	32
Rysunek 3.2: Diagram czynności – inicjalizacja połączenia i menu główne (Wariant A, DTMF)	34
Rysunek 3.3: Diagram czynności – uwierzytelnianie pacjenta (Wariant A, DTMF)	35
Rysunek 3.4: Diagram czynności – rejestracja wizyty (Wariant A, DTMF)	36
Rysunek 3.5: Diagram czynności – inicjalizacja połączenia i rozpoznanie intencji (Wariant B, NLU)	38
Rysunek 3.6: Diagram czynności – uwierzytelnianie pacjenta (Wariant B, NLU)	39
Rysunek 3.7: Diagram czynności – rejestracja wizyty (Wariant B, NLU)	40
Rysunek 3.8: Diagram Architektury systemu – widok komponentów i integracji	44

Spis tabel

Tabela 2.1: Analiza porównawcza klasycznych systemów IVR i botów konwersacyjnych [7].	11
Tabela 2.2: Analiza porównawcza platform Amazon Connect Customer i Twilio Flex.	15
Tabela 2.3: Zestawienie najważniejszych ograniczeń platformy Amazon Connect [18].	18
Tabela 2.4: Rola usług AWS w architekturze systemu CCaaS [19].	23
Tabela 2.5: Wymagania dotyczące obsługi połączeń przychodzących.....	25
Tabela 2.6: Wymagania dotyczące informacji ogólnych funkcjonowania placówki.....	26
Tabela 2.7: Wymagania dotyczące obsługi pacjenta.....	26
Tabela 2.8: Wymagania dotyczące obsługi przez agenta.....	27
Tabela 2.9: Wymagania niefunkcjonalne rozważanego systemu CCaaS.....	28
Tabela 3.1: Specyfikacja intencji bota konwersacyjnego Amazon Lex V2.....	41